Einsatz verifizierter Architekturen

BeeGFS on NetApp with E-Series Storage

NetApp
March 21, 2024

This PDF was generated from https://docs.netapp.com/de-de/beegfs/beegfs-solution-overview.html on March 21, 2024. Always check docs.netapp.com for the latest.

Inhalt

Einsatz verifizierter Architekturen 1
Überblick und Anforderungen 1
Überprüfen des Lösungsdesigns 10
Implementieren der Lösung 24

Einsatz verifizierter Architekturen

Überblick und Anforderungen

Lösungsüberblick

Die BeeGFS auf NetApp Lösung kombiniert das parallele BeeGFS Filesystem mit NetApp EF600 Storage-Systemen und bietet so eine zuverlässige, skalierbare und kostengünstige Infrastruktur, die mit anspruchsvollen Workloads Schritt hält.

Dieses Design nutzt die Performance-Dichte, die von neuester Enterprise Server- und Storage-Hardware und Netzwerkgeschwindigkeiten bereitgestellt wird, Da File-Nodes mit dualen AMD EPYC 7003 „Mailand“-Prozessoren ausgestattet sind und PCIe 4.0 mit Direktverbindung über 200 GB (HDR) InfiniBand zu Block-Nodes unterstützt werden, die End-to-End-NVMe und NVMeOF unter Verwendung des NVMe/IB-Protokolls bieten.

NVA-Programm

Die BeeGFS on NetApp Lösung ist Teil des NetApp Verified Architecture (NVA) Programms, das Kunden Referenzkonfigurationen und Orientierungshilfen zur Größenbestimmung für spezifische Workloads und Anwendungsfälle bietet. NVA-Lösungen werden ausführlich getestet und entwickelt, um Implementierungsrisiken zu minimieren und die Markteinführungszeit zu verkürzen.

Anwendungsfälle

Die folgenden Anwendungsfälle gelten für die BeeGFS auf NetApp Lösung:

- Künstliche Intelligenz (KI), einschließlich Machine Learning (ML), Deep Learning (DL), großzügiger natürlicher Sprachverarbeitung (NLP) und NLU (Natural Language Understanding) Weitere Informationen finden Sie unter "BeeGFS for AI: Fakt versus Fiction".
- High-Performance Computing (HPC) einschließlich Applikationen, die mit MPI (Message Passing Interface) und anderen Distributed Computing-Techniken beschleunigt werden. Weitere Informationen finden Sie unter "Warum BeeGFS das HPC übertrifft".
- Applikations-Workloads zeichnen sich durch folgende Merkmale aus:
  - Lesen oder Schreiben auf Dateien mit einer Größe von mehr als 1 GB
  - Lesen oder Schreiben in dieselbe Datei durch mehrere Clients (10s, 100s und 1000s)
- Datensätze mit mehreren Terabyte oder mehreren Petabyte.
- Umgebungen, für die ein einziger Storage Namespace benötigt wird, der sich für eine Mischung aus großen und kleinen Dateien optimieren lässt

Vorteile

Die wichtigsten Vorteile von BeeGFS auf NetApp:

- Verfügbarkeit verifizierter Hardware-Designs mit vollständiger Integration von Hardware- und Softwarekomponenten, die zuverlässige Performance und Zuverlässigkeit gewährleisten.
- Implementierung und Management mit Ansible für Einfachheit und Konsistenz nach Maß
- Überwachung und Beobachtbarkeit mithilfe des E-Series Performance Analyzer und BeeGFS Plug-ins.

Weitere Informationen finden Sie unter "Framework zur Überwachung von NetApp E-Series Lösungen".

- Hochverfügbarkeit dank einer Shared-Disk-Architektur für Datenaufbewahrungszeit und -Verfügbarkeit
- Unterstützung für modernes Workload-Management und moderne Orchestrierung mithilfe von Containern und Kubernetes Weitere Informationen finden Sie unter "Kubernetes Meet BeeGFS: Eine Geschichte zukunftssichere Investition".

HA-Architektur

BeeGFS auf NetApp erweitert die Funktionalität der BeeGFS Enterprise Edition durch Entwicklung einer vollständig integrierten Lösung mit NetApp Hardware, die eine HA-Architektur (Shared Disk High Availability, Shared-Hochverfügbarkeit) ermöglicht.

> Die BeeGFS Community Edition kann zwar kostenlos genutzt werden, jedoch muss bei der Enterprise Edition ein Professional Support-Abonnementvertrag von einem Partner wie NetApp abgeschlossen werden. Die Enterprise-Version ermöglicht die Nutzung mehrerer zusätzlicher Funktionen wie Ausfallsicherheit, Kontingentzuverfolgung und Storage-Pools.

In der folgenden Abbildung werden die HA-Architekturen ohne Shared-Festplatten verglichen.

[]

Weitere Informationen finden Sie unter "Ankündigung der Hochverfügbarkeit für BeeGFS mit Unterstützung von NetApp".

Ansible

BeeGFS auf NetApp wird mittels Ansible-Automatisierung bereitgestellt und implementiert. Das Hosting wird auf GitHub und Ansible Galaxy (die BeeGFS-Sammlung ist über verfügbar "Ansible-Galaxie" Und "NetApp E-Series GitHub"). Obwohl Ansible vor allem mit der Hardware getestet wird, die zum Zusammenbauen der BeeGFS-Bausteine verwendet wird, können Sie es so konfigurieren, dass es auf nahezu jedem x86-basierten Server unter Verwendung einer unterstützten Linux-Distribution ausgeführt wird.

Weitere Informationen finden Sie unter "Implementieren von BeeGFS mit E-Series Storage".

Generationen im Design

Die BeeGFS on NetApp-Lösung befindet sich derzeit in der zweiten Generation.

Sowohl die erste als auch die zweite Generation verfügen über eine Basisarchitektur mit einem BeeGFS Filesystem und einem NVMe EF600 Storage-System. Die zweite Generation baut jedoch auf der ersten auf und beinhaltet folgende zusätzliche Vorteile:

- Verdoppeln Sie die Performance und Kapazität bei gleichzeitiger Ergänzung von nur 2 HE Rack-Stellfläche
- Hochverfügbarkeit (HA) auf Grundlage eines gemeinsam genutzten zwei-Tier-Hardwaredesigns
- Externe Qualifizierung für NVIDIA DGX A100 SuperPOD und NVIDIA BasePOD Architekturen

Design der zweiten Generation

Die zweite Generation von BeeGFS auf NetApp ist für die Performance-Anforderungen anspruchsvoller Workloads optimiert, darunter High-Performance-Computing (HPC) und HPC-ähnliche Techniken für Machine Learning (ML), Deep Learning (DL) und ähnliche Techniken für künstliche Intelligenz (KI). Durch die Integration einer HA-Architektur (Shared Disk High Availability, Hochverfügbarkeit) erfüllt die BeeGFS auf NetApp Lösung

auch die Anforderungen von Unternehmen und anderen Unternehmen, die sich keine Ausfallzeiten oder Datenverluste leisten können, wenn es um Storage geht, der durch Skalierung mit ihren Workloads und Anwendungsfällen Schritt halten kann. Diese Lösung wurde nicht nur von NetApp verifiziert, sondern bestand auch die externe Qualifizierung als Storage-Option für NVIDIA DGX SuperPOD und DGX BasePOD.

Design der ersten Generationen

Die erste Generation von BeeGFS auf NetApp wurde für Machine-Learning- (ML) und AI-Workloads (künstliche Intelligenz) mit NetApp EF600 NVMe-Storage-Systemen, dem BeeGFS Parallel-Filesystem, NVIDIA DGX™ A100-Systemen und NVIDIA® Mellanox® Quantum™ QM8700 IB-Switches mit 200 Gbit/s entwickelt. Dieses Design verfügt außerdem über 200 Gbit/s InfiniBand (IB) für die Storage- und Compute-Cluster Interconnect-Fabric und stellt damit eine vollständig auf IB basierende Architektur für hochperformante Workloads bereit.

Weitere Informationen zur ersten Generation finden Sie unter "NetApp EF-Series AI mit NVIDIA DGX A100 Systems und BeeGFS".

Überblick über die Architektur

Die BeeGFS on NetApp Lösung beinhaltet Design-Aspekte, die bei der Architekturentwicklung berücksichtigt werden, um die spezifischen Geräte, Kabel und Konfigurationen zu ermitteln, die für validierte Workloads erforderlich sind.

Modulare Architektur

Das BeeGFS-Dateisystem kann je nach Storage-Anforderungen unterschiedlich implementiert und skaliert werden. In Anwendungsfällen, die in erster Linie mehrere kleine Dateien enthalten, profitieren beispielsweise von der zusätzlichen Performance und Kapazität der Metadaten, während in Anwendungsfällen mit weniger großen Dateien mehr Storage-Kapazität und Performance für die tatsächlichen Dateiinhalte erforderlich wären. Diese verschiedenen Überlegungen wirken sich auf die verschiedenen Dimensionen der Implementierung paralleler Dateisysteme aus, was die Entwicklung und Implementierung des Filesystems weiter vereinfacht.

Zur Bewältigung dieser Herausforderungen hat NetApp eine standardmäßige Bausteinarchitektur entwickelt, mit der sich jede dieser Dimensionen skalieren lässt. BeeGFS-Bausteine werden in der Regel in einem von drei Konfigurationsprofilen bereitgestellt:

- Ein einzelner Baustein, einschließlich BeeGFS-Management, Metadaten und Storage-Services
- Ein BeeGFS Metadaten plus Storage-Baustein
- Ein BeeGFS-Lagergebäude

Die einzige Hardware-Änderung zwischen diesen drei Optionen ist die Verwendung kleinerer Laufwerke für BeeGFS-Metadaten. Andernfalls werden alle Konfigurationsänderungen durch die Software übernommen. Und mit Ansible als Implementierungs-Engine gestaltet sich die Einrichtung des gewünschten Profils für einen bestimmten Baustein die Konfigurationsaufgaben unkompliziert.

Weitere Informationen finden Sie unter Verifiziertes Hardwaredesign.

File-System-Services

Das BeeGFS-Dateisystem umfasst die folgenden Hauptdienste:

- **Management Service.** registriert und überwacht alle anderen Dienste.

- **Speicherdienst.** speichert den verteilten Inhalt der Benutzerdatei, bekannt als Datenblock-Dateien.
- **Metadatendienst.** verfolgt das Dateisystem-Layout, Verzeichnis, Dateiattribute und so weiter.
- **Client Service.** installiert das Dateisystem, um auf die gespeicherten Daten zuzugreifen.

Die folgende Abbildung zeigt die Komponenten und Beziehungen der BeeGFS-Lösung für NetApp E-Series Systeme.

[]

Als paralleles Dateisystem verteilt BeeGFS seine Dateien auf mehrere Server-Nodes, um die Lese-/Schreib-Performance und Skalierbarkeit zu maximieren. Die Server-Knoten arbeiten zusammen, um ein einziges Dateisystem bereitzustellen, das gleichzeitig von anderen Server-Knoten, allgemein bekannt als *Clients*, gemountet werden kann. Diese Clients können das verteilte Dateisystem auf ähnliche Weise wie ein lokales Dateisystem wie NTFS, XFS oder ext4 sehen und nutzen.

Die vier wichtigsten Services werden in einer Vielzahl von unterstützten Linux Distributionen ausgeführt und kommunizieren über jedes TCP/IP- oder RDMA-fähige Netzwerk, einschließlich InfiniBand (IB), Omni-Path (OPA) und RDMA over Converged Ethernet (RoCE). Die BeeGFS Server Services (Management, Speicherung und Metadaten) sind Benutzerspace-Dämonen, während der Client ein natives Kernel-Modul (patchless) ist. Alle Komponenten können ohne Neustart installiert oder aktualisiert werden. Sie können beliebige Kombinationen von Services auf demselben Node ausführen.

Verifizierte Nodes

Die BeeGFS auf NetApp Lösung umfasst die folgenden verifizierten Knoten: Das NetApp EF600 Storage-System (Block-Node) und der Lenovo ThinkSystem SR665 Server (File-Node).

Block-Node: EF600 Storage-System

Das NetApp EF600 All-Flash-Array bietet konsistenten Datenzugriff nahezu in Echtzeit und unterstützt gleichzeitig eine beliebige Anzahl von Workloads. Damit Daten schnell und kontinuierlich in KI- und HPC-Applikationen eingespeist werden können, sorgen EF600 Storage-Systeme für bis zu zwei Millionen Lese-IOPS im Cache, Reaktionszeiten von unter 100 Mikrosekunden und eine sequenzielle Lesebandbreite von 42 GB/s in einem Gehäuse.

Dateiknoten: Lenovo ThinkSystem SR665 Server

Der SR665 ist ein 2-HE-Server mit zwei Sockets und PCIe 4.0. Wenn sie so konfiguriert ist, dass sie die Anforderungen dieser Lösung erfüllt, bietet sie genügend Performance für die Ausführung von BeeGFS-Fileservices in einer Konfiguration, die mit der Verfügbarkeit des Durchsatzes und der IOPS-Werte der direkt angeschlossenen E-Series Nodes ausgeglichen ist.

Weitere Informationen zum Lenovo SR665 finden Sie unter "Lenovo Website".

Verifiziertes Hardwaredesign

Die Bausteine der Lösung (in der folgenden Abbildung dargestellt) verwenden zwei PCIe-4.0-fähige Dual-Socket-Server für die BeeGFS-File-Schicht und zwei EF600 Storage-Systeme als Block-Schicht.

[]

> Da jeder Baustein zwei BeeGFS-Datei-Nodes enthält, sind mindestens zwei Bausteine erforderlich, um im Failover-Cluster ein Quorum einzurichten. Zwar können Sie ein Cluster mit zwei Nodes konfigurieren, jedoch verfügt diese Konfiguration über Grenzwerte, die möglicherweise ein erfolgreiches Failover verhindern. Wenn Sie ein 2-Node-Cluster benötigen, können Sie ein drittes Gerät als Tiebreaker einbinden (dieses Design ist jedoch nicht an diesem Standort abgedeckt).

Jedes Bausteinsystem bietet Hochverfügbarkeit durch ein zweistufigen Hardware-Design, das Fehlerdomänen für die Datei- und Block-Ebenen trennt. Jede Tier kann unabhängig ausfallen, was die Ausfallsicherheit erhöht und das Risiko von kaskadierenden Ausfällen minimiert. Die Verwendung von HDR InfiniBand in Verbindung mit NVMeOF ermöglicht einen hohen Durchsatz und eine minimale Latenz zwischen Datei- und Block-Nodes, wobei volle Redundanz und eine ausreichende Überzeichnung von Links möglich ist, um zu einem Engpass zu führen, auch wenn das System teilweise beeinträchtigt wird.

Die BeeGFS on NetApp Lösung läuft über alle Bausteine während der Implementierung hinweg. Im ersten bereitgestellten Baustein muss BeeGFS-Management, Metadaten und Storage-Services (als Basisbaustein bezeichnet) ausgeführt werden. Alle nachfolgenden Bausteine werden über Software konfiguriert, um BeeGFS Metadaten und Storage-Services auszuführen, oder nur Storage-Services. Die Verfügbarkeit verschiedener Konfigurationsprofile für jeden Baustein ermöglicht die Skalierung von Filesystem-Metadaten oder Storage-Kapazität und Performance unter Verwendung derselben zugrunde liegenden Hardware-Plattformen und desselben Block-Designs.

Bis zu fünf Bausteine werden zu einem eigenständigen Linux HA-Cluster kombiniert, wodurch eine angemessene Anzahl von Ressourcen pro Cluster Resource Manager (Pacemaker) sichergestellt wird und der Messaging-Overhead reduziert wird, der erforderlich ist, um Cluster-Mitglieder synchron zu halten (Corosync). Pro Cluster wird mindestens zwei Bausteine empfohlen, damit genügend Mitglieder Quorum festlegen können. Ein oder mehrere dieser eigenständigen BeeGFS HA-Cluster werden kombiniert, um ein BeeGFS-Dateisystem zu erstellen (siehe in der folgenden Abbildung), auf das Clients als einzelner Storage-Namespace zugreifen können.

[]

Obwohl die Anzahl der Bausteine pro Rack letztendlich von den Energie- und Kühlungsanforderungen für einen bestimmten Standort abhängt, Die Lösung wurde so entwickelt, dass bis zu fünf Bausteine in einem einzelnen 42-HE-Rack implementiert werden können und gleichzeitig noch Platz für zwei 1-HE-InfiniBand-Switches für das Storage-/Datennetzwerk vorhanden ist. Jeder Baustein benötigt acht IB-Ports (vier pro Switch für Redundanz), sodass fünf Bausteine die Hälfte der Ports an einem 40-Port-HDR-InfiniBand-Switch (wie das NVIDIA QM8700) zur Implementierung einer FAT-Tree- oder ähnlichen Nonblocking-Topologie übrig lassen. So wird sichergestellt, dass sich die Anzahl der Storage- bzw. Compute-/GPU-Racks ohne Netzwerkengpässe aufskalieren lässt. Eine überzeichnete Storage-Fabric kann optional auf Empfehlung des Storage-Fabric-Anbieters verwendet werden.

Das folgende Bild zeigt eine 80-Node-FAT-Tree-Topologie.

[]

Mithilfe von Ansible als Implementierungs-Engine für die Implementierung von BeeGFS auf NetApp können Administratoren die gesamte Umgebung mit moderner Infrastruktur als Code-Practice verwalten. Dies vereinfacht drastisch, was ein komplexes System ansonsten bedeuten würde. Administratoren können so Konfigurationen an einem Ort definieren und anpassen und anschließend unabhängig von der Größe der Umgebung konsistent anwenden. Die BeeGFS Kollektion ist erhältlich ab "Ansible-Galaxie" Und "NetApp E-Series GitHub".

Technische Anforderungen

Für die Implementierung der BeeGFS auf NetApp Lösung stellen Sie sicher, dass Ihre Umgebung die technologischen Anforderungen erfüllt.

Hardwareanforderungen

In der folgenden Tabelle sind die Hardwarekomponenten aufgeführt, die für die Implementierung eines Bausteindesigns der zweiten Generation für die BeeGFS auf NetApp Lösung erforderlich sind.

> Je nach den Anforderungen des Kunden können die in einer bestimmten Implementierung dieser Lösung verwendeten Hardwarekomponenten abweichen.

Zählen	Hardwarekomponenten	Anforderungen
2	BeeGFS-Datei-Knoten.	Jeder Datei-Node sollte die folgende Konfiguration erfüllen oder übertreffen, um die erwartete Performance zu erreichen. • Prozessoren:* • 2x AMD EPYC 7343 16C 3.2 GHz. • Konfiguriert als zwei NUMA-Zonen. **Speicher:** • 256 GB: • 16x 16 GB TruDDR4 3200 MHz (2Rx8 1,2 V) RDIMM-A (eher kleinere DIMMs als weniger größere DIMMs). • Gefüllt, um die Speicherbandbreite zu maximieren. **PCIe Erweiterung: Vier PCE Gen4 x16 Steckplätze:** • Zwei Steckplätze pro NUMA-Zone. • Jeder Steckplatz sollte ausreichend Strom-/Kühlleistung für den Mellanox MCX653106A-HDAT-Adapter bereitstellen. **Verschiedenes:** • Zwei 1-TB-SATA-Laufwerke mit 7.200 U/min (oder vergleichbar), konfiguriert in RAID 1 für das Betriebssystem. • 10 GbE OCP 3.0 Adapter (oder vergleichbar) für in-Band-BS-Management • 1GbE BMC mit Redfish API für Out-of-Band-Server-Management • Zwei Hot-Swap-fähige Netzteile und Performance-Lüfter • Zum Erreichen von Storage InfiniBand Switches müssen Mellanox optische InfiniBand-Kabel unterstützt werden. • Lenovo SR665:* • Das benutzerdefinierte NetApp Modell enthält die erforderliche Version der XClarity-Controller-Firmware, die zur Unterstützung von Mellanox ConnectX-6-Adaptern mit zwei Ports erforderlich ist. Wenden Sie sich an NetApp, um Bestellinformationen zu erhalten.
8	Mellanox ConnectX-6 HCAs (für File-Nodes)	• MCX653106A-HDAT Host Channel Adapter (HDR IB 200 GB, Dual-Port QSFP56, PCIe4.0 x16).
8	1-m-HDR-InfiniBand-Kabel (für direkte Verbindungen mit Datei-/Block-Nodes)	• MCP1650-H001E30 (1 m Mellanox Passives Kupferkabel, IB HDR, bis zu 200 Gbit/s, QSFP56, 30 AWG). Die Länge kann angepasst werden, um bei Bedarf längere Distanzen zwischen Datei- und Block-Knoten zu berücksichtigen.

Zählen	Hardwarekomponenten	Anforderungen
8	HDR-InfiniBand-Kabel (für File-Node-/Storage-Switch-Verbindungen)	Benötigt InfiniBand HDR-Kabel (QSFP56-Transceiver) von der entsprechenden Länge, um Datei-Knoten mit Speicher-Leaf-Switches zu verbinden. Mögliche Optionen sind: • MCP1650-H002E26 (2 m Mellanox Passives Kupferkabel, IB HDR, bis 200 GB/s, QSFP56, 30 AWG). • MFS1S00-H003E (3 m Mellanox-Aktivfaserkabel, IB-HDR, bis zu 200 GB/s, QSFP56).
2	Block-Nodes der E-Series	Zwei EF600 Controller werden wie folgt konfiguriert: • Arbeitsspeicher: 256 GB (128 GB pro Controller) • Adapter: 2-Port 200 GB/HDR (NVMe/IB). • Laufwerke: Passend zur gewünschten Kapazität konfiguriert.

Softwareanforderungen

Um eine vorhersehbare Performance und Zuverlässigkeit zu gewährleisten, werden Releases der BeeGFS auf NetApp Lösung mit spezifischen Versionen der Softwarekomponenten getestet, die für die Implementierung der Lösung erforderlich sind.

Anforderungen an die Softwareimplementierung

In der folgenden Tabelle sind die automatisch bereitgestellten Softwareanforderungen im Rahmen der Ansible-basierten BeeGFS-Implementierung aufgeführt.

Software	Version
BeeGFS	7.2.6
Corosync	3.1.5-1
Schrittmacher	2.1.0-8
OpenSM	opensm-5.9.0 (von mlnx_ofed 5.4-1.0.3.0) ⓘ Nur erforderlich für die direkte Verbindung zur Virtualisierung

Ansible-Control-Node-Anforderungen

Die BeeGFS auf NetApp Lösung wird über einen Ansible-Kontroll-Node implementiert und gemanagt. Weitere Informationen finden Sie im "Ansible-Dokumentation".

Die in den folgenden Tabellen aufgeführten Software-Anforderungen beziehen sich speziell auf die unten aufgeführte Version der NetApp BeeGFS Ansible Sammlung.

Software	Version
Ansible	2.11 bei Installation über pip: ansible-4.7.0 und ansible-Core < 2.12,>=2.11.6

Software	Version
Python	3.9
Zusätzliche Python-Pakete	Kryptografie-35.0.0, netaddr-0.8.0
BeeGFS Ansible Collection	3.0.0

Anforderungen an Datei-Nodes

Software	Version
Red hat Enterprise Linux	Redhat 8.4 Server, physisch mit Hochverfügbarkeit (2 Sockets). File-Nodes erfordern ein gültiges RedHat Enterprise Linux Server-Abonnement und das Red hat Enterprise Linux High Availability Add-On.
Linux-Kernel	4.18.0-305.25.1.el8_4.x86_64
InfiniBand-/RDMA-Treiber	Posteingang
ConnectX-6 HCA-Firmware	FW: 20.31.1014
PXE: 3.6.0403	UEFI: 14.24.0013

Anforderungen der EF600 Block-Nodes

Software	Version
SANtricity OS	11.70.2
NVSRAM	N6000-872834-D06.dlp
Festplatten-Firmware	Neueste verfügbar für die verwendeten Antriebsmodelle.

Zusätzlichen Anforderungen

Die in der folgenden Tabelle aufgeführten Geräte wurden zur Validierung verwendet, es können jedoch bei Bedarf geeignete Alternativen verwendet werden. In der Regel empfiehlt NetApp die Ausführung der neuesten Softwareversion, um unerwartete Probleme zu vermeiden.

Hardwarekomponenten	Installierte Software
• 2 x Mellanox MQM8700 200-GB-InfiniBand-Switches	• Firmware 3.9.2110

Hardwarekomponenten	Installierte Software
1x Ansible-Steuerungsknoten (virtualisiert): • Prozessoren: Intel® Xeon® Gold 6146 CPU @ 3,20 GHz • Speicher: 8 GB • Lokaler Storage: 24 GB	• CentOS Linux 8.4.2105 • Kernel 4.18.0-305.3.1.el8.x86_64 Installierte Ansible- und Python-Versionen entsprechen den in der oben stehenden Tabelle.
10x BeeGFS Clients (CPU Nodes): • Prozessor: 1x AMD EPYC 7302 16-Core CPU bei 3,0 GHz • Speicher: 128 GB • Netzwerk: 2x Mellanox MCX653106A-HDAT (ein Port pro Adapter angeschlossen).	• Ubuntu 20.04 • Kernel: 5.4.0-100-generic • InfiniBand-Treiber: Mellanox OFED 5.4-1.0.3.0
1x BeeGFS Client (GPU-Knoten): • Prozessoren: 2x AMD EPYC 7742 64-Core CPUs bei 2,25 GHz • Speicher: 1 TB • Netzwerk: 2x Mellanox MCX653106A-HDAT (ein Port pro Adapter angeschlossen). Dieses System basiert auf der NVIDIAs HGX A100 Plattform und umfasst vier A100 GPUs.	• Ubuntu 20.04 • Kernel: 5.4.0-100-generic • InfiniBand-Treiber: Mellanox OFED 5.4-1.0.3.0

Überprüfen des Lösungsdesigns

Designübersicht

Spezifische Geräte, Kabel und Konfigurationen sind erforderlich, um die BeeGFS auf NetApp Lösung zu unterstützen, die das parallele Filesystem BeeGFS mit den NetApp EF600 Storage-Systemen kombiniert.

Weitere Informationen:

- "Hardwarekonfiguration"
- "Softwarekonfiguration"
- "Design-Überprüfung"
- "Richtlinien für die Dimensionierung"
- "Performance-Optimierung"

Derivative Architekturen mit Variationen in Design und Performance:

- "Baustein Mit Hoher Kapazität"

Hardwarekonfiguration

Die Hardware-Konfiguration für BeeGFS auf NetApp umfasst Datei-Nodes und Netzwerkverkabelung.

Konfiguration der Datei-Nodes

File-Nodes haben zwei CPU-Sockets als separate NUMA-Zonen konfiguriert, die lokalen Zugriff auf eine gleiche Anzahl von PCIe-Steckplätzen und Arbeitsspeicher enthalten.

InfiniBand-Adapter müssen in die entsprechenden PCI-Risers oder Steckplätze gefüllt sein, damit die Workload über die verfügbaren PCIe-Lanes und Speicherkanäle ausgeglichen ist. Sie balancieren den Workload aus, indem einzelne BeeGFS-Services vollständig auf einen bestimmten NUMA-Node isoliert werden. Das Ziel besteht darin, bei jedem Datei-Node eine ähnliche Performance zu erreichen, als ob es sich um zwei unabhängige Single-Socket-Server handelte.

Die folgende Abbildung zeigt die NUMA-Konfiguration des Dateiknotens.

[]

Die BeeGFS-Prozesse sind an eine bestimmte NUMA-Zone gebunden, um sicherzustellen, dass sich die verwendeten Schnittstellen in der gleichen Zone befinden. Diese Konfiguration vermeidet den Remote-Zugriff über die Verbindung zwischen den Sockets. Die Verbindung zwischen den Sockeln wird manchmal als QPI oder GMI2 Link bezeichnet; selbst in modernen Prozessorarchitekturen können sie einen Engpass darstellen, wenn High-Speed-Netzwerke wie HDR InfiniBand eingesetzt werden.

Konfiguration der Netzwerkverkabelung

Jeder Datei-Node ist innerhalb eines Bausteins mit zwei Block-Nodes verbunden. Dabei werden insgesamt vier redundante InfiniBand-Verbindungen verwendet. Zusätzlich verfügt jeder Datei-Node über vier redundante Verbindungen zum InfiniBand-Storage-Netzwerk.

Beachten Sie in der folgenden Abbildung Folgendes:

- Alle grün dargestellten Datei-Node-Ports werden zur Verbindung mit der Storage-Fabric verwendet. Alle anderen Datei-Node-Ports sind die direkten Verbindungen zu den Block-Nodes.
- Zwei InfiniBand-Ports in einer bestimmten NUMA-Zone werden mit den A- und B-Controllern desselben Block-Nodes verbunden.
- Ports im NUMA-Knoten 0 stellen immer eine Verbindung zum ersten Block-Knoten her.
- Die Ports im NUMA-Knoten 1 verbinden sich mit dem zweiten Block-Knoten.

[]

> Bei Speichernetzwerken mit redundanten Switches sollten die in hellgrün dargestellten Ports mit einem Switch verbunden werden. Die in Dunkelgrün-Ports sollten mit einem anderen Switch verbunden werden.

Die in der Abbildung dargestellte Verkabelungskonfiguration ermöglicht jedem BeeGFS-Dienst Folgendes:

- Laufen Sie in derselben NUMA-Zone, unabhängig davon, auf welchem Dateiknoten der BeeGFS-Service ausgeführt wird.
- Sekundäre optimale Pfade zum Front-End-Storage-Netzwerk und zu den Back-End-Block-Nodes sind

vorhanden, unabhängig davon, wo ein Ausfall auftritt.

- Minimale Auswirkungen auf die Performance, wenn ein Datei-Node oder Controller in einem Block-Node gewartet werden muss

Verkabelung zur Nutzung der Bandbreite

Um die vollständige bidirektionale PCIe-Bandbreite zu nutzen, stellen Sie sicher, dass ein Port an jedem InfiniBand-Adapter mit der Storage-Fabric verbunden ist, und der andere Port mit einem Block-Node verbunden wird. Die theoretische Maximalgeschwindigkeit eines HDR InfiniBand-Ports beträgt 25 Gbit/s (nicht für Signalisierung und andere Gemeinkosten). Die maximale Bandbreite für eine Richtung in einem PCIe 4.0 x16-Steckplatz beträgt 32 Gbit/s und führt zu einem potenziellen Engpass bei der Implementierung von Datei-Nodes mit InfiniBand-Adaptern mit zwei Ports, die theoretisch 50 Gbit/s Bandbreite verarbeiten können.

Die folgende Abbildung zeigt das Verkabelungsdesign, mit dem die vollständige bidirektionale PCIe-Bandbreite genutzt werden kann.

[]

Verwenden Sie für jeden BeeGFS-Service denselben Adapter, um den bevorzugten Port für Client-Datenverkehr mit dem Pfad zu dem Block-Nodes-Controller zu verbinden, der der primäre Eigentümer dieser Service-Volumes ist. Weitere Informationen finden Sie unter "Softwarekonfiguration".

Softwarekonfiguration

Die Software-Konfiguration für BeeGFS auf NetApp umfasst BeeGFS-Netzwerkkomponenten, EF600 Block-Nodes, BeeGFS-Datei-Nodes, Ressourcengruppen und BeeGFS-Services.

BeeGFS-Netzwerkkonfiguration

Die BeeGFS-Netzwerkkonfiguration besteht aus den folgenden Komponenten.

- **Schwimmende IPs** schwimmende IPs sind eine Art virtueller IP-Adresse, die dynamisch an jeden Server im selben Netzwerk weitergeleitet werden kann. Mehrere Server können dieselbe Floating-IP-Adresse besitzen, sie kann jedoch nur auf einem Server zu einem bestimmten Zeitpunkt aktiv sein.

  Jeder BeeGFS-Serverdienst hat eine eigene IP-Adresse, die sich je nach Ausführungsort des BeeGFS-Serverdienstes zwischen Datei-Nodes verschieben kann. Diese fließende IP-Konfiguration ermöglicht jedem Service ein unabhängiges Failover auf den anderen File-Node. Der Client muss einfach die IP-Adresse für einen bestimmten BeeGFS-Service kennen. Es muss nicht wissen, welcher Datei-Node derzeit diesen Service ausführt.

- **BeeGFS-Server Multi-Homing-Konfiguration** um die Dichte der Lösung zu erhöhen, verfügt jeder Dateiknoten über mehrere Speicherschnittstellen mit IPs, die im gleichen IP-Subnetz konfiguriert sind.

  Es ist eine zusätzliche Konfiguration erforderlich, um sicherzustellen, dass diese Konfiguration wie erwartet mit dem Linux-Netzwerk-Stack funktioniert, da standardmäßig Anfragen an eine Schnittstelle auf einer anderen Schnittstelle beantwortet werden können, wenn ihre IPs im selben Subnetz sind. Neben anderen Nachteilen ist es bei diesem Standardverhalten unmöglich, RDMA-Verbindungen ordnungsgemäß einzurichten oder zu pflegen.

  Die Ansible-basierte Implementierung verarbeitet das Anziehen des Reverse Path (RP) und das ARP-Verhalten (Address Resolution Protocol) und stellt sicher, dass Floating IPs gestartet und gestoppt werden. Die entsprechenden IP-Routen und -Regeln werden dynamisch erstellt, damit die Multihomed-

Netzwerkkonfiguration ordnungsgemäß funktioniert.

- **BeeGFS Client Multi-Rail-Konfiguration** *Multi-Rail* bezieht sich auf die Fähigkeit einer Anwendung, mehrere unabhängige Netzwerk-„Rails“ zu verwenden, um die Leistung zu steigern.

  BeeGFS kann zwar RDMA für die Konnektivität verwenden, doch nutzt BeeGFS IPoIB, um die Erkennung und Einrichtung von RDMA-Verbindungen zu vereinfachen. Damit BeeGFS-Clients mehrere InfiniBand-Schnittstellen verwenden können, können Sie jeden Client mit einer IP-Adresse in einem anderen Subnetz konfigurieren und dann die bevorzugten Schnittstellen für die Hälfte der BeeGFS-Serverdienste in jedem Subnetz konfigurieren.

  In der folgenden Abbildung sind hellgrün hervorgehobene Schnittstellen in einem IP-Subnetz (z. B. `100.127.0.0/16`) Und die dunkelgrünen Schnittstellen befinden sich in einem anderen Subnetz (z.B. `100.128.0.0/16`).

  Die folgende Abbildung zeigt die Verteilung des Datenverkehrs auf mehrere BeeGFS-Client-Schnittstellen.

  []

  Da jede Datei in BeeGFS normalerweise über mehrere Storage-Services verteilt wird, kann der Client dank der Multi-Rail-Konfiguration einen höheren Durchsatz erzielen als mit einem einzelnen InfiniBand-Port möglich. Die folgende Codeprobe zeigt beispielsweise eine allgemeine File-Striping-Konfiguration, die es dem Client ermöglicht, den Datenverkehr über beide Schnittstellen auszugleichen:

```
root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
Entry type: file
EntryID: 11D-624759A9-65
Metadata node: meta_01_tgt_0101 [ID: 101]
Stripe pattern details:
+ Type: RAID0
+ Chunksize: 1M
+ Number of storage targets: desired: 4; actual: 4
+ Storage targets:
  + 101 @ stor_01_tgt_0101 [ID: 101]
  + 102 @ stor_01_tgt_0101 [ID: 101]
  + 201 @ stor_02_tgt_0201 [ID: 201]
  + 202 @ stor_02_tgt_0201 [ID: 201]
```

  Die Verwendung von zwei IPoIB-Subnetzen ist ein logischer Unterschied. Sie können bei Bedarf ein einzelnes physisches InfiniBand Subnetz (Speichernetzwerk) verwenden.

> In BeeGFS 7.3.0 wurde Multi-Rail-Unterstützung hinzugefügt, um die Verwendung mehrerer IB-Schnittstellen in einem IPoIB-Subnetz zu ermöglichen. Das Design für die BeeGFS auf NetApp Lösung wurde vor der allgemeinen Verfügbarkeit von BeeGFS 7.3 entwickelt und demonstriert somit den Einsatz von zwei IP-Subnetzen für die Nutzung von zwei IB-Schnittstellen auf den BeeGFS-Clients. Ein Vorteil des Ansatzes mit mehreren IP-Subnetzen besteht darin, dass Multihoming auf BeeGFS-Client-Nodes nicht mehr konfiguriert werden muss (weitere Informationen finden Sie unter "BeeGFS RDMA-Unterstützung").

Konfiguration der EF600 Block-Node

Block-Nodes bestehen aus zwei aktiv/aktiv-RAID-Controllern mit gemeinsamem Zugriff auf denselben Satz an Laufwerken. In der Regel besitzt jeder Controller die Hälfte der im System konfigurierten Volumes, jedoch kann für den anderen Controller nach Bedarf die Aufgaben übernehmen.

Multipathing-Software auf den Datei-Knoten bestimmt den aktiven und optimierten Pfad zu jedem Volume und verschiebt sich automatisch nach einem Kabel-, Adapter- oder Controller-Ausfall zum alternativen Pfad.

Das folgende Diagramm zeigt das Controller-Layout in EF600 Block-Nodes.

[]

Um eine Shared-Disk-HA-Lösung zu erleichtern, werden Volumes beiden Datei-Nodes zugeordnet, sodass sie sich bei Bedarf gegenseitig übernehmen können. Das folgende Diagramm zeigt ein Beispiel, wie BeeGFS-Service und die bevorzugte Volumeneigentümer für maximale Performance konfiguriert sind. Die Schnittstelle links von jedem BeeGFS-Dienst gibt die bevorzugte Schnittstelle an, mit der die Clients und andere Dienste Kontakt aufnehmen.

[]

Im vorherigen Beispiel kommunizieren Clients und Server Services lieber mit Storage Service 1 über Schnittstelle i1b. Storage Service 1 verwendet Schnittstelle i1a als bevorzugten Pfad zur Kommunikation mit seinen Volumes (Storage_tgt_101, 102) auf Controller A des ersten Block-Node. Diese Konfiguration nutzt die volle bidirektionale PCIe-Bandbreite, die dem InfiniBand-Adapter zur Verfügung steht, und erreicht mit einem Dual-Port-HDR-InfiniBand-Adapter eine bessere Leistung als bei PCIe 4.0.

BeeGFS-Dateiknoten-Konfiguration

Die BeeGFS Datei-Nodes sind in einem HA-Cluster (High Availability) konfiguriert, um den Failover von BeeGFS-Services zwischen mehreren Datei-Nodes zu ermöglichen.

Das HA Cluster Design basiert auf zwei weit verbreiteten Linux HA Projekten: Corosync für Cluster-Mitgliedschaft und Pacemaker für Cluster Resource Management. Weitere Informationen finden Sie unter "Red hat Training für Add-ons mit hoher Verfügbarkeit".

NetApp hat mehrere Open Cluster Framework (OCF) Resource Agents entwickelt und erweitert, damit das Cluster die BeeGFS Ressourcen intelligent starten und überwachen kann.

BeeGFS HA Cluster

Wenn Sie einen BeeGFS-Dienst starten (mit oder ohne HA), müssen in der Regel einige Ressourcen vorhanden sein:

- IP-Adressen, in denen der Dienst erreichbar ist, werden normalerweise von Network Manager konfiguriert.
- Zugrunde liegende Dateisysteme, die als Ziele für BeeGFS zum Speichern von Daten verwendet werden.

  Diese werden typischerweise in definiert `/etc/fstab` Und montiert von systemd.

- Ein systemd-Service, der für den Start von BeeGFS-Prozessen verantwortlich ist, wenn die anderen Ressourcen bereit sind.

  Ohne zusätzliche Software werden diese Ressourcen nur auf einem einzelnen Datei-Node gestartet. Wenn der Datei-Node offline geschaltet wird, kann auf einen Teil des BeeGFS-Dateisystems zugegriffen werden.

Da mehrere Nodes jeden BeeGFS-Service starten können, muss Pacemaker sicherstellen, dass jeder Service und die abhängigen Ressourcen nur auf jeweils einem Node ausgeführt werden. Wenn beispielsweise zwei Knoten versuchen, denselben BeeGFS-Service zu starten, besteht das Risiko einer Datenbeschädigung, wenn beide versuchen, auf dieselben Dateien auf dem zugrunde liegenden Ziel zu schreiben. Um dieses Szenario zu vermeiden, setzt Pacemaker auf Corosync, um den Zustand des gesamten Clusters zuverlässig über alle Knoten hinweg zu synchronisieren und Quorum zu schaffen.

Wenn ein Fehler im Cluster auftritt, reagiert Pacemaker und startet BeeGFS-Ressourcen auf einem anderen Knoten neu. In einigen Fällen kann Pacemaker möglicherweise nicht mit dem ursprünglichen fehlerhaften Knoten kommunizieren, um zu bestätigen, dass die Ressourcen angehalten werden. Um zu überprüfen, ob der Knoten ausgefallen ist, bevor BeeGFS-Ressourcen an anderer Stelle neu gestartet werden, isoliert Pacemaker den fehlerhaften Knoten, idealerweise indem er die Stromversorgung entfernt.

Es stehen zahlreiche Open-Source-Fechten-Agenten zur Verfügung, die es Pacemaker ermöglichen, einen Knoten mit einer Stromverteilungs-Einheit (PDU) oder den Server-Baseboard-Management-Controller (BMC) mit APIs wie Redfish zu Zaun.

Wenn BeeGFS in einem HA-Cluster ausgeführt wird, werden alle BeeGFS-Services und zugrunde liegenden Ressourcen von Pacemaker in Ressourcengruppen gemanagt. Jeder BeeGFS-Service und die Ressourcen, auf die er angewiesen ist, werden in einer Ressourcengruppe konfiguriert, die sicherstellt, dass Ressourcen in der richtigen Reihenfolge gestartet und gestoppt werden und auf demselben Node zusammengelegt werden.

Für jede BeeGFS-Ressourcengruppe führt Pacemaker eine benutzerdefinierte BeeGFS-Überwachungsressource aus, die für die Erkennung von Fehlerbedingungen und die intelligente Auslösung von Failover verantwortlich ist, wenn auf einem bestimmten Knoten kein BeeGFS-Dienst mehr verfügbar ist.

Die folgende Abbildung zeigt die Pacemaker-gesteuerten BeeGFS-Dienste und -Abhängigkeiten.

[]

> Damit mehrere BeeGFS-Dienste desselben Typs auf demselben Knoten gestartet werden, wird Pacemaker so konfiguriert, dass BeeGFS-Dienste mit der Multi-Mode-Konfigurationsmethode gestartet werden. Weitere Informationen finden Sie im "BeeGFS-Dokumentation im Multi-Modus".

Da BeeGFS-Dienste auf mehreren Nodes starten können müssen, muss die Konfigurationsdatei für jeden Dienst (normalerweise bei gefunden `/etc/beegfs`) Wird auf einem der E-Series Volumes gespeichert, die als BeeGFS-Ziel für diesen Service verwendet werden. Damit sind die Konfiguration zusammen mit den Daten für einen bestimmten BeeGFS Service für alle Nodes zugänglich, die den Service möglicherweise ausführen müssen.

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

Design-Überprüfung

Das Design der zweiten Generation für die BeeGFS auf NetApp Lösung wurde mithilfe von drei Bausteinkonfigurationsprofilen verifiziert.

Die Konfigurationsprofile umfassen Folgendes:

- Ein einzelner Baustein, einschließlich BeeGFS-Management, Metadaten und Storage-Services
- Ein BeeGFS Metadaten plus ein Storage-Baustein.
- Ein BeeGFS-Speicherbaustein.

Die Bausteine wurden mit zwei Mellanox Quantum InfiniBand-Switches (MQM8700) verbunden. Zehn BeeGFS Clients wurden auch an die InfiniBand Switches angeschlossen und zur Ausführung von Synthetic Benchmark Utilities verwendet.

Die folgende Abbildung zeigt die BeeGFS-Konfiguration, die zur Validierung der BeeGFS auf NetApp Lösung verwendet wird.

[]

BeeGFS-Datei-Striping

Ein Vorteil paralleler Dateisysteme besteht darin, einzelne Dateien über mehrere Storage-Ziele zu verteilen, wodurch Volumes auf demselben oder verschiedenen zugrunde liegenden Storage-Systemen dargestellt werden können.

In BeeGFS können Sie Striping auf Verzeichnisbasis und pro Datei konfigurieren, um die Anzahl der für jede Datei verwendeten Ziele zu steuern und die für jeden Dateistripe verwendete Chunksize (oder Blockgröße) zu steuern. Bei dieser Konfiguration kann das Filesystem verschiedene Workload- und I/O-Profile unterstützen, ohne Services neu konfigurieren oder neu starten zu müssen. Sie können Stripe-Einstellungen mit dem

anwenden `beegfs-ctl` Befehlszeilen-Tool oder Anwendungen, die die Striping-API verwenden. Weitere Informationen finden Sie in der BeeGFS-Dokumentation für "Striping" Und "Striping-API".

Um eine optimale Leistung zu erzielen, wurden während des Tests Streifenmuster angepasst und die für jeden Test verwendeten Parameter werden notiert.

IOR-Bandbreitentests: Mehrere Clients

Die IOR-Bandbreitentests verwendeten OpenMPI, um parallele Jobs des synthetischen E/A-Generatorwerkzeugs IOR auszuführen (verfügbar unter "HPC GitHub") Über alle 10 Client-Knoten zu einem oder mehreren BeeGFS-Bausteinen. Sofern nicht anders angegeben:

- Alle Tests verwendeten einen direkten I/O mit einer Übertragungsgröße von 1 MiB.
- BeeGFS-Datei-Striping wurde auf 1 MB Chunksize und ein Ziel pro Datei eingestellt.

Für IOR wurden die folgenden Parameter verwendet, wobei die Segmentanzahl angepasst wurde, um die aggregierte Dateigröße für einen Baustein auf 5 tib und 40 tib für drei Bausteine zu halten.

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

Baustein für eine BeeGFS-Basis (Management, Metadaten und Storage

Die folgende Abbildung zeigt die IOR-Testergebnisse mit einem einzelnen BeeGFS-Basisspeicher (Management, Metadaten und Storage).

[]

BeeGFS Metadaten + Storage-Baustein

Die folgende Abbildung zeigt die IOR-Testergebnisse mit einem einzelnen BeeGFS-Metadaten + einem Storage-Baustein.

[]

BeeGFS-Speicherbaustein

Die folgende Abbildung zeigt die IOR-Testergebnisse mit einem einzelnen BeeGFS-Storage-only-Baustein.

[]

Drei BeeGFS-Bausteine

Die folgende Abbildung zeigt die IOR-Testergebnisse mit drei BeeGFS-Bausteinen.

[]

Wie erwartet, ist der Performance-Unterschied zwischen dem Basis-Baustein und den nachfolgenden Metadaten + dem Storage-Baustein vernachlässigbar. Ein Vergleich zwischen Metadaten und Storage-Bausteinen und einem ausschließlich Storage-Baustein zeigt einen leichten Anstieg der Lese-Performance aufgrund der zusätzlichen Laufwerke, die als Storage-Ziele verwendet werden. Allerdings gibt es keinen wesentlichen Unterschied in der Schreib-Performance. Um eine höhere Performance zu erzielen, können Sie mehrere Bausteine gleichzeitig hinzufügen, um die Performance linear zu skalieren.

IOR-Bandbreitentests: Einzelner Client

Der IOR-Bandbreitentest nutzte OpenMPI, um mehrere IOR-Prozesse mithilfe eines einzigen leistungsstarken GPU-Servers auszuführen, um die Performance zu untersuchen, die für einen einzelnen Client erreichbar ist.

Dieser Test vergleicht auch das Verhalten und die Leistung von BeeGFS, wenn der Client so konfiguriert ist, dass er den Linux-Kernel-Page-Cache verwendet (`tuneFileCacheType = native`) Im Vergleich zum Standard `buffered` Einstellung.

Der native Caching-Modus verwendet den Linux-Kernel-Page-Cache auf dem Client, sodass die Readervorgänge nicht über das Netzwerk übertragen werden, sondern aus dem lokalen Speicher stammen.

Das folgende Diagramm zeigt die IOR-Testergebnisse mit drei BeeGFS-Bausteinen und einem einzelnen Client.

[]

> BeeGFS Striping für diese Tests wurde auf 1 MB Chunksize mit acht Zielen pro Datei eingestellt.

Obwohl die Performance bei den Schreibzugriffen und beim ersten Lesen im standardmäßigen gepufferten Modus höher ist, werden bei Workloads, die dieselben Daten mehrmals lesen, eine deutliche Performance-Steigerung im nativen Caching-Modus erzielt. Diese verbesserte Performance bei erneuten Lesevorgängen ist für Workloads wie Deep Learning wichtig, die denselben Datensatz mehrmals in vielen Epoch-Durchläufen lesen.

Metadaten-Performance-Test

Bei den Metadaten-Performance-Tests wurde das MDTest-Tool (im Rahmen von IOR enthalten) verwendet, um die Metadaten-Performance von BeeGFS zu messen. Die Tests verwendeten OpenMPI, um parallele Jobs auf allen zehn Client-Knoten auszuführen.

Die folgenden Parameter wurden verwendet, um den Benchmark-Test mit der Gesamtzahl der Prozesse von 10 auf 320 in Schritt 2 x und mit einer Dateigröße von 4k durchgeführt.

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

Die Metadaten-Performance wurde zuerst mit ein bis zwei Metadaten + Storage-Bausteinen gemessen, um die Performance durch das Hinzufügen weiterer Bausteine zu verdeutlichen.

1 BeeGFS Metadaten + Storage-Baustein

Das folgende Diagramm zeigt die MDTest-Ergebnisse mit einer BeeGFS-Metadaten + Speicherbausteinen.

[]

Zwei BeeGFS Metadaten + Storage-Bausteine

Das folgende Diagramm zeigt die MDTest-Ergebnisse mit zwei BeeGFS-Metadaten + Speicherbausteinen.

[]

Funktionsprüfung

Im Rahmen der Validierung dieser Architektur führte NetApp mehrere Funktionstests durch, darunter:

- Ausfall eines einzelnen InfiniBand-Ports des Clients durch Deaktivieren des Switch-Ports
- Ausfall eines InfiniBand-Ports mit einem einzelnen Server durch Deaktivieren des Switch-Ports
- Sofortige Abschaltung des Servers mithilfe des BMC.
- Anmutig Platzierung eines Node im Standby-Modus und Failover-Betrieb zu einem anderen Node
- Anmutig Setzen eines Node wieder online und Failback-Services auf den ursprünglichen Node.
- Schalten Sie einen der InfiniBand-Switches mithilfe der PDU aus. Alle Tests wurden durchgeführt, während Belastungstests mit dem durchgeführt wurden `sysSessionChecksEnabled: false` Parameter auf BeeGFS-Clients gesetzt. Es wurden keine Fehler oder Störungen bei I/O festgestellt.

> Es ist ein bekanntes Problem aufgetreten (siehe "Changelog") Wenn BeeGFS-Client/Server-RDMA-Verbindungen unerwartet unterbrochen werden, entweder durch Ausfall der primären Schnittstelle (wie in definiert `connInterfacesFile`) Oder ein BeeGFS-Server fällt aus. Aktive Client-I/O kann bis zu zehn Minuten lang aufhängen, bevor der Vorgang fortgesetzt wird. Dieses Problem tritt nicht auf, wenn BeeGFS-Knoten ordnungsgemäß für geplante Wartung in den Standby-Modus versetzt oder TCP verwendet wird.

NVIDIA DGX A100 SuperPOD- und BasePOD-Validierung

NetApp validierte eine Storage-Lösung für IANVIDDGX A100 SuperPOD unter Verwendung eines ähnlichen BeeGFS Filesystem, das aus drei Bausteinen mit den Metadaten und angewandtem Storage-Konfigurationsprofil besteht. Die Qualifizierung bestand darin, die von dieser NVA beschriebene Lösung mit zwanzig DGX A100 GPU-Servern zu testen, auf denen eine Vielzahl von Storage-, Machine-Learning- und Deep-Learning-Benchmarks ausgeführt wurden. Der gesamte Storage, der für die Verwendung in NVIDIA DGX A100 SuperPOD zertifiziert ist, ist ebenfalls automatisch für die Verwendung in NVIDIA BasePOD-Architekturen zertifiziert.

Weitere Informationen finden Sie unter "NVIDIA DGX SuperPOD mit NetApp" Und "NVIDIA DGX BasePOD".

Richtlinien für die Dimensionierung

Die BeeGFS Lösung enthält Empfehlungen für das Performance- und Kapazitäts-Sizing, die auf Verifizierungstests basieren.

Die Bausteinarchitektur verfolgt das Ziel, eine Lösung zu entwickeln, die sich einfach dimensionieren lässt. Dazu werden mehrere Bausteine hinzugefügt, die die Anforderungen für ein bestimmtes BeeGFS-System erfüllen. Mithilfe der nachstehenden Richtlinien können Sie die Anzahl und die Arten von BeeGFS-Bausteinen schätzen, die für die Anforderungen Ihrer Umgebung benötigt werden.

Beachten Sie, dass es sich bei diesen Schätzungen um die beste Performance handelt. Benchmark-Applikationen werden geschrieben und verwendet, um die Nutzung der zugrunde liegenden Filesysteme auf eine Weise zu optimieren, die reale Applikationen vielleicht nicht erreichen.

Performance-Dimensionierung

Die folgende Tabelle enthält ein empfohlene Performance-Sizing.

Konfigurationsprofil	1 MiB Lesevorgänge	1 MiB Schreibvorgänge
Metadaten + Storage	62Gibps	21Gibps
Nur Storage	64Gibps	21Gibps

Die Schätzungen zur Größe der Metadaten-Kapazität basieren auf der „Faustregel“, dass eine Kapazität von 500 GB für etwa 150 Millionen Dateien in BeeGFS ausreichend ist. (Weitere Informationen finden Sie in der BeeGFS-Dokumentation für "Systemanforderungen".)

Die Verwendung von Funktionen wie Zugriffssteuerungslisten und die Anzahl der Verzeichnisse und Dateien pro Verzeichnis wirkt sich auch darauf aus, wie schnell der Metadatenspeicherplatz verbraucht wird. Schätzungen zur Storage-Kapazität berücksichtigen neben RAID 6 und XFS-Overhead die nutzbare Laufwerkskapazität.

Kapazitätsdimensionierung für Metadaten + Storage-Bausteine

Die folgende Tabelle enthält eine empfohlene Kapazitätsdimensionierung für Metadaten und Storage-Bausteine.

Laufwerksgröße (2+2 RAID 1) Metadaten-Volume-Gruppen	Metadaten-Kapazität (Anzahl der Dateien)	Laufwerksgröße (8+2 RAID 6) Storage-Volume-Gruppen	Speicherkapazität (File-Inhalte)
1,92 TB	1,938,577,200	1,92 TB	51,77 TB
3,84 TB	3,880,388,400	3,84 TB	103,5 TB
7,68 TB	8,125,278,000	7,68 TB	216,74 TB
15,3 TB	17,269,854,000	15,3 TB	460 TB

Bei der Größenbestimmung von Metadaten und Storage-Bausteinen können die Kosten reduziert werden, da weniger Laufwerke für Metadaten-Volume-Gruppen anstelle von Storage-Volume-Gruppen verwendet werden.

Kapazitätsdimensionierung für reine Storage-Bausteine

Die folgende Tabelle zeigt die Größenanpassung der Kapazität für reine Storage-Bausteine.

Laufwerksgröße (10+2 RAID 6) Storage-Volume-Gruppen	Speicherkapazität (File-Inhalte)
1,92 TB	59,89 TB
3,84 TB	119,80 TB
7,68 TB	251,89 TB
15,3 TB	538,5 TB

Die Performance- und der Kapazitäts-Overhead, die durch das Einbinden des Management-Service in den Basis-Baustein (erster) verursacht werden, sind minimal, es sei denn, die globale Dateisperrung ist aktiviert.

Performance-Optimierung

Die BeeGFS-Lösung enthält Empfehlungen für Performance-Tuning, die auf Verifikationstests basieren.

Obwohl BeeGFS Out-of-the-Box-Performance bietet, hat NetApp eine Reihe von empfohlenen Tuning-Parametern entwickelt, um die Performance zu maximieren. Diese Parameter berücksichtigen die Funktionen der zugrunde liegenden Block-Nodes der E-Series und alle speziellen Anforderungen, die für die Ausführung von BeeGFS in einer HA-Architektur mit Shared-Festplatten erforderlich sind.

Performance-Tuning für Datei-Nodes

Zu den verfügbaren Tuning-Parametern, die Sie konfigurieren können, gehören folgende:

1. **Systemeinstellungen im UEFI/BIOS von Datei-Nodes.** um die Leistung zu maximieren, empfehlen wir die Konfiguration der Systemeinstellungen auf dem Server-Modell, das Sie als Datei-Knoten verwenden. Sie konfigurieren die Systemeinstellungen, wenn Sie die Datei-Nodes einrichten, indem Sie entweder das System-Setup (UEFI/BIOS) oder die Redfish-APIs verwenden, die vom Baseboard-Management-Controller (BMC) bereitgestellt werden.

   Die Systemeinstellungen variieren je nach Servermodell, das Sie als Dateiknoten verwenden. Die Einstellungen müssen manuell auf der Grundlage des verwendeten Servermodells konfiguriert werden. Informationen zur Konfiguration der Systemeinstellungen für die validierten Lenovo SR665-Dateiknoten finden Sie unter "Optimieren Sie die System-Einstellungen des File Node für die Performance".

2. **Standardeinstellungen für erforderliche Konfigurationsparameter.** die erforderlichen Konfigurationsparameter beeinflussen die Konfiguration von BeeGFS-Diensten und die Formatierung und Bereitstellung von E-Series-Volumes (Blockgeräte) durch Pacemaker. Die folgenden Konfigurationsparameter sind erforderlich:

   - BeeGFS Service-Konfigurationsparameter

     Sie können die Standardeinstellungen für die Konfigurationsparameter bei Bedarf überschreiben. Informationen zu den Parametern, die Sie sich für Ihre spezifischen Workloads oder Anwendungsfälle anpassen können, finden Sie im "Parameter für BeeGFS-Service-Konfiguration".

   - Für die Volume-Formatierung und die Montage-Parameter gelten die empfohlenen Standardeinstellungen, die nur für erweiterte Anwendungsfälle angepasst werden sollten. Die Standardwerte führen folgende Schritte aus:

     - Optimieren Sie die ursprüngliche Volume-Formatierung auf Basis des Zieltyps (beispielsweise Management, Metadaten oder Storage), zusammen mit der RAID-Konfiguration und Segmentgröße des zugrunde liegenden Volumes.
     - Passen Sie an, wie Pacemaker die einzelnen Lautstärkerahmen einstellt, um sicherzustellen, dass Änderungen sofort an Block-Nodes der E-Series gespült werden. So wird Datenverlust verhindert, wenn bei aktiven Schreibvorgängen Datei-Nodes ausfallen.

       Informationen zu den Parametern, die Sie sich für Ihre spezifischen Workloads oder Anwendungsfälle anpassen können, finden Sie im "Volume-Formatierung und Montage der Konfigurationsparameter".

3. **Systemeinstellungen im Linux-Betriebssystem, das auf den Dateiknoten installiert ist.** Sie können die standardmäßigen Linux-Betriebssystemeinstellungen überschreiben, wenn Sie den Ansible-Bestand in Schritt 4 von erstellen "Erstellen des Ansible-Inventars".

Die Standardeinstellungen wurden zur Validierung der BeeGFS auf NetApp Lösung verwendet. Sie können sie jedoch an Ihre spezifischen Workloads oder Anwendungsfälle anpassen. Einige Beispiele für die Linux-Betriebssystemeinstellungen, die Sie ändern können, sind:

- I/O-Warteschlangen an Block-Geräten der E-Series.

  Auf den als BeeGFS-Ziele verwendeten E-Series-Block-Geräten können Sie I/O-Warteschlangen folgendermaßen konfigurieren:

  - Passen Sie den Planungsalgorithmus basierend auf dem Gerätetyp (NVMe, HDD usw.) an.
  - Erhöhen Sie die Anzahl der ausstehenden Anfragen.
  - Passen Sie Anfragegrößen an.
  - Optimierung des Verhaltens beim Lesen.
- Einstellungen für virtuellen Speicher.

  Sie können die Einstellungen für den virtuellen Speicher für eine optimale Streaming-Leistung anpassen.

- CPU-Einstellungen.

  Sie können den CPU-Frequenzregler und andere CPU-Konfigurationen für maximale Leistung anpassen.

- Anfragegröße lesen

  Sie können die maximale Lese-Anfraegröße für Mellanox HCAs erhöhen.

Performance-Tuning für Block-Nodes

Basierend auf den Konfigurationsprofilen, die auf einen bestimmten BeeGFS-Baustein angewendet werden, ändern sich die auf den Block-Nodes konfigurierten Volume-Gruppen leicht. Beispiel mit einem EF600 Block-Node mit 24 Laufwerken:

- Für den einzelnen Basis-Block, einschließlich BeeGFS-Management, Metadaten und Storage-Services:
  - 1x 2+2 RAID 10 Volume-Gruppe für BeeGFS Management und Metadaten-Services
  - 2 x 8+2 RAID 6-Volume-Gruppen für BeeGFS-Storage-Services
- Für BeeGFS Metadaten + Storage-Baustein:
  - 1x 2+2 RAID 10 Volume-Gruppe für BeeGFS Metadaten-Services
  - 2 x 8+2 RAID 6-Volume-Gruppen für BeeGFS-Storage-Services
- Nur für BeeGFS-Lagergebäude:
  - 2 x 10+2 RAID 6-Volume-Gruppen für BeeGFS-Storage-Services

> Da BeeGFS für Management und Metadaten im Gegensatz zu Storage erheblich weniger Speicherplatz benötigt, besteht die Möglichkeit, für die RAID 10-Volume-Gruppen kleinere Laufwerke zu verwenden. Kleinere Laufwerke sollten in den äußeren Laufwerksteckplätzen befüllt werden. Weitere Informationen finden Sie im "Implementierungsanleitungen".

Diese werden alle durch die Ansible-basierte Implementierung konfiguriert, und verschiedene andere allgemein empfohlene Einstellungen für Performance-/Verhaltensoptimierung:

- Anpassung der globalen Cache-Blockgröße auf 32 KiB und Anpassung der bedarfsorientierten Cache-Flush auf 80 %.
- Deaktivieren des automatischen Load-Balancing (sicherstellen, dass die Controller-Volume-Zuweisungen wie vorgesehen bleiben).
- Aktivieren von Lese-Caching und Deaktivieren des Read-Ahead-Caching
- Aktivieren des Schreib-Caches mit Spiegelung und Bedarf an Akku-Backups, sodass Caches während des Ausfalls eines Block-Node-Controllers bestehen.
- Festlegen der Reihenfolge, in der Laufwerke Volume-Gruppen zugewiesen werden, und Ausgleich der I/O-Vorgänge über verfügbare Laufwerkskanäle

Baustein mit hoher Kapazität

Bei der Entwicklung der BeeGFS-Standardlösung wurde auf Workloads mit hoher Performance gedenkt. Kunden, die nach Anwendungsfällen mit hoher Kapazität suchen, sollten die hier beschriebenen Design- und Performance-Unterschiede beobachten.

Hardware- und Softwarekonfiguration

Hardware- und Softwarekonfiguration für den Baustein mit hoher Kapazität ist Standard. Es sei denn, die EF600 Controller sollten durch einen EF300 Controller ersetzt werden. Sie können zwischen 1 und 7 IOM-Erweiterungsfächern mit jeweils 60 Laufwerken pro Storage-Array anschließen, Insgesamt 2 bis 14 Erweiterungsfächer pro Baustein.

Unternehmen, die ein High-Capacity-Baustein-Design implementieren, verwenden wahrscheinlich nur die Basis-Bausteinkonfiguration, die aus BeeGFS-Management, Metadaten und Storage-Services für jeden Node besteht. Um die Kosteneffizienz zu steigern, sollten Storage-Nodes mit hoher Kapazität Metadaten-Volumes auf den NVMe-Laufwerken im EF300-Controller-Gehäuse bereitstellen und Storage-Volumes für die NL-SAS-Laufwerke in den Erweiterungsfächern bereitstellen.

[]

Richtlinien für die Dimensionierung

Bei diesen Richtlinien zur Dimensionierung ist davon auszugehen, dass Bausteine mit hoher Kapazität mit einer NVMe-SSD-Volume-Gruppe von 2+2 für Metadaten im Basis-EF300-Gehäuse und 6 x 8+2 NL-SAS-Volume-Gruppen pro IOM-Erweiterungsfach für Storage konfiguriert sind.

Laufwerkgröße (Kapazitäts-HDDs)	Kapazität pro BB (1 Fach)	Kapazität pro BB (2 Einschübe)	Kapazität pro BB (3 Einschübe)	Kapazität pro BB (4 Einschübe)
4 TB	439 TB	878 TB	1317 TB	1756 TB
8 TB	878 TB	1756 TB	2634 TB	3512 TB
10 TB	1097 TB	2195 TB	3292 TB	4390 TB
12 TB	1317 TB	2634 TB	3951 TB	5268 TB
16 TB	1756 TB	3512 TB	5268 TB	7024 TB
18 TB	1975 TB	3951 TB	5927 TB	7902 TB

Implementieren der Lösung

Implementierungsübersicht

Mit BeeGFS können Sie auf NetApp bereitstellen und Datei- und Block-Nodes mithilfe der zweiten Generation des NetApp BeeGFS-Bausteindesigns validieren.

Ansible-Sammlungen und -Funktionen

Ansible implementiert die BeeGFS auf NetApp Lösung, eine beliebte IT-Automatisierungs-Engine, mit der Applikationsimplementierungen automatisiert werden. Ansible verwendet eine Reihe von Dateien, die gemeinsam als Inventar bezeichnet werden und mit denen das BeeGFS-Dateisystem, das bereitgestellt werden soll, modelliert wird.

Ansible ermöglicht Unternehmen wie NetApp, mit Sammlungen auf Ansible Galaxy (siehe) die integrierte Funktionalität auszubauen "NetApp E-Series BeeGFS Sammlung"). Die Sammlungen umfassen Module, die bestimmte Funktionen oder Aufgaben ausführen (wie z. B. ein E-Series Volume erstellen) und enthalten Rollen, die mehrere Module und andere Rollen aufrufen können. Dieser automatisierte Ansatz reduziert die Zeit für die Implementierung des BeeGFS-Filesystems und des zugrunde liegenden HA-Clusters. Zudem vereinfacht es das Hinzufügen von Bausteinen, mit denen die bestehenden Filesysteme erweitert werden können.

Weitere Informationen finden Sie unter "Weitere Informationen zum Ansible Inventar".

> Da zahlreiche Schritte an der Implementierung der BeeGFS auf NetApp-Lösung beteiligt sind, unterstützt NetApp die manuelle Bereitstellung der Lösung nicht.

Konfigurationsprofile für BeeGFS-Bausteine

Die Implementierungsverfahren umfassen die folgenden Konfigurationsprofile:

- Ein einziger Baustein, der Management-, Metadaten- und Storage-Services umfasst
- Ein zweiter Baustein, der Metadaten und Storage-Services umfasst.
- Ein dritter Baustein, der nur Storage-Services umfasst.

Diese Profile veranschaulichen die gesamte Palette der empfohlenen Konfigurationsprofile für die NetApp BeeGFS-Bausteine. Für jede Implementierung kann die Anzahl der Metadaten und Storage-Bausteine oder Storage-Services-only Bausteine in den Prozeduren je nach Kapazitäts- und Performance-Anforderungen variieren.

Übersicht über die einzelnen Implementierungsschritte

Die Bereitstellung umfasst folgende allgemeine Aufgaben:

Hardwarebereitstellung

1. Stellen Sie jeden Baustein physisch zusammen.
2. Rack-und Kabelhardware: Ausführliche Verfahren finden Sie unter "Implementierung von Hardware".

Softwareimplementierung

1. "Richten Sie Datei- und Block-Nodes ein".

    - Konfigurieren Sie BMC-IPs auf Datei-Knoten
    - Installieren Sie ein unterstütztes Betriebssystem und konfigurieren Sie Managementnetzwerk auf Datei-Knoten
    - Konfiguration der Management-IPs auf Block-Nodes
2. "Richten Sie einen Ansible-Steuerungsknoten ein".
3. "Passen Sie die Systemeinstellungen für die Performance an".
4. "Erstellen des Ansible-Inventars".
5. "Definieren Sie den Ansible-Bestand für BeeGFS-Bausteine".
6. "Implementieren Sie BeeGFS mit Ansible".
7. "Konfigurieren Sie BeeGFS-Clients".

> Die Bereitstellungsverfahren umfassen mehrere Beispiele, in denen Text in eine Datei kopiert werden muss. Achten Sie bei allen Inline-Kommentaren, die mit „#“- oder „/“-Zeichen gekennzeichnet sind, auf alles, was für eine bestimmte Bereitstellung geändert werden sollte oder werden kann. Beispiel:
>
> ```
> beegfs_ha_ntp_server_pools: # THIS IS AN EXAMPLE OF A COMMENT!
> - "pool 0.pool.ntp.org iburst maxsources 3"
> - "pool 1.pool.ntp.org iburst maxsources 3"
> ```

Derivative Architekturen mit Variationen bei Implementierungsempfehlungen:

- "Baustein Mit Hoher Kapazität"

Weitere Informationen zum Ansible Inventar

Bevor Sie mit der Implementierung beginnen, sollten Sie wissen, wie Sie Ansible verwenden können, um die BeeGFS auf NetApp Lösung mit dem BeeGFS-Bausteindesign der zweiten Generation zu konfigurieren und zu implementieren.

Ansible-Bestandsaufnahme definiert die Konfiguration für Datei- und Block-Nodes und stellt das BeeGFS-Dateisystem dar, das bereitgestellt werden soll. Der Bestand umfasst Hosts, Gruppen und Variablen, die das gewünschte BeeGFS-Dateisystem beschreiben. Probenbestände können von heruntergeladen werden "NetApp E-Series BeeGFS GitHub".

Ansible-Module und -Rollen

Verwenden Sie zum Anwenden der im Ansible-Inventar beschriebenen Konfiguration die verschiedenen Ansible-Module und -Rollen, die in der NetApp E-Series Ansible Sammlung bereitgestellt werden, insbesondere die BeeGFS HA 7.2-Rolle (über die erhältlich "NetApp E-Series BeeGFS GitHub") Das die End-to-End-Lösung implementiert.

Jede Rolle der NetApp E-Series Ansible Sammlung ist eine vollständige End-to-End-Implementierung der BeeGFS auf NetApp Lösung. Die Rollen verwenden die Sammlungen NetApp E-Series SANtricity, Host und BeeGFS, mit denen Sie das BeeGFS Filesystem mit HA (High Availability, Hochverfügbarkeit) konfigurieren können. Anschließend können Sie Storage bereitstellen und zuordnen und den Cluster-Storage betriebsbereit machen.

Die Rollen verfügen über eine ausführliche Dokumentation. In den Implementierungsverfahren wird beschrieben, wie die Rolle bei der Implementierung einer NetApp Verified Architecture mit dem BeeGFS-

Bausteindesign der zweiten Generation eingesetzt wird.

> Obwohl die Implementierungsschritte versucht, genügend Details bereitzustellen, sodass frühere Erfahrungen mit Ansible nicht erforderlich sind, sollten Sie mit Ansible und der zugehörigen Terminologie vertraut sein.

Bestandslayout für BeeGFS HA-Cluster

Verwenden Sie die Ansible-Bestandsstruktur, um ein BeeGFS HA-Cluster zu definieren.

Jeder, der über frühere Ansible-Erfahrungen verfügt, sollte beachten, dass die BeeGFS HA-Rolle eine benutzerdefinierte Methode zur Ermittlung der Variablen (oder Fakten) für jeden Host implementiert. Dies ist erforderlich, um den Aufbau eines Ansible-Bestands zu vereinfachen, in dem Ressourcen beschrieben werden, die auf mehreren Servern ausgeführt werden können.

Ein Ansible-Inventar besteht in der Regel aus den Dateien in `host_vars` Und `group_vars`, Und an `inventory.yml` Datei, die Hosts bestimmten Gruppen (und möglicherweise Gruppen zu anderen Gruppen) zuweist.

> Erstellen Sie keine Dateien mit dem Inhalt in diesem Unterabschnitt, der nur als Beispiel gedacht ist.

Obwohl diese Konfiguration anhand des Konfigurationsprofils vorab festgelegt ist, sollten Sie wie folgt allgemeine Kenntnisse darüber haben, wie alles als Ansible-Inventar ausgelegt ist:

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
        meta_01:  # Group representing a metadata service with ID 01.
          hosts:
            file_node_01:  # This service is preferred on the first file
node.
            file_node_02:  # And can failover to the second file node.
        meta_02:  # Group representing a metadata service with ID 02.
          hosts:
            file_node_02:  # This service is preferred on the second file
node.
            file_node_01: # And can failover to the first file node.
```

Für jeden Dienst wird unter eine zusätzliche Datei erstellt `group_vars` Beschreibung der Konfiguration:

```
# meta_01 - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: 8015
  connMetaPortUDP: 8015
  tuneBindToNumaZone: 0
floating_ips:
  - i1b: <IP>/<SUBNET_MASK>
  - i4b: <IP>/<SUBNET_MASK>
# Type of BeeGFS service the HA resource group will manage.
beegfs_service: metadata # Choices: management, metadata, storage.
# What block node should be used to create a volume for this service:
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        owning_controller: A
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.25
```

Mit diesem Layout können BeeGFS-Service-, Netzwerk- und Storage-Konfigurationen für jede Ressource an einem Ort definiert werden. Hinter den Kulissen aggregiert die Rolle BeeGFS basierend auf dieser Bestandsstruktur die erforderliche Konfiguration für jede Datei und jeden Block-Node. Weitere Informationen finden Sie in diesem Blog-Beitrag: "NetApp beschleunigt die Implementierung von HA für BeeGFS mit Ansible".

> Die numerische BeeGFS- und String-Node-ID für jeden Dienst wird automatisch auf Basis des Gruppennamens konfiguriert. Zusätzlich zur allgemeinen Ansible-Anforderung, dass Gruppennamen eindeutig sein sollen, müssen Gruppen, die einen BeeGFS-Service darstellen, in einer Zahl enden, die für den BeeGFS-Service eindeutig ist, für den diese Gruppe repräsentiert. Zum Beispiel sind meta_01 und stor_01 zulässig, aber Metadaten_01 und meta_01 sind nicht.

Besprechen der Best Practices

Beachten Sie bei der Implementierung der BeeGFS auf NetApp-Lösung die Best Practice-Richtlinien.

Standardkonventionen

Folgen Sie beim physischen Zusammenbau und Erstellen der Ansible-Bestandsdatei diesen Standardkonventionen (weitere Informationen finden Sie unter "Erstellen des Ansible-Inventars").

- Host-Namen der Dateiknoten werden nacheinander nummeriert (h01-HN) mit niedrigeren Zahlen oben im Rack und höheren Zahlen unten.

Beispielsweise die Namenskonvention `[location][row][rack]hN` Sieht aus wie: `ictad22h01`.

- Jeder Block-Node besteht aus zwei Storage-Controllern, die jeweils über einen eigenen Host-Namen verfügen.

  Mit einem Storage-Array-Namen wird im Rahmen eines Ansible-Inventars das gesamte Block-Storage-System bezeichnet. Die Namen des Speicher-Arrays sollten nacheinander nummeriert sein (a01 - an), und die Hostnamen für einzelne Controller werden aus dieser Namenskonvention abgeleitet.

  Beispiel: Ein Block-Node mit dem Namen `ictad22a01` Normalerweise können Host-Namen für jeden Controller wie konfiguriert sein `ictad22a01-a` Und `ictad22a01-b`, Aber als Ansible-Inventar bezeichnet werden `ictad22a01`.

- File- und Block-Nodes innerhalb desselben Bausteins teilen sich das gleiche Nummerierungsschema und sind im Rack nebeneinander, wobei beide Datei-Nodes oben und beide Block-Nodes direkt darunter liegen.

  Im ersten Baustein sind beispielsweise die Datei-Nodes h01 und h02 direkt mit den Block-Nodes a01 und a02 verbunden. Von oben nach unten sind die Hostnamen h01, h02, a01 und a02.

- Bausteine werden in sequenzieller Reihenfolge auf der Grundlage ihrer Hostnamen installiert, sodass sich die niedrigeren Host-Namen oben im Rack befinden und die höheren nummerierten Host-Namen sich unten befinden.

  Ziel ist es, die Länge des Kabels zu minimieren, das oben auf den Rack Switches läuft, und eine standardisierte Implementierungspraxis zu definieren, um die Fehlerbehebung zu vereinfachen. Für Rechenzentren, in denen dies nicht erlaubt ist, aufgrund von Bedenken um die Rack-Stabilität, ist die umgekehrte sicherlich erlaubt, das Befüllen des Racks von unten nach oben.

InfiniBand-Storage-Netzwerkkonfiguration

Die Hälfte der InfiniBand-Ports an jedem Datei-Node werden für eine direkte Verbindung mit Block-Nodes verwendet. Die andere Hälfte ist mit den InfiniBand-Switches verbunden und wird für die BeeGFS-Client-Server-Konnektivität verwendet. Beim Bestimmen der Größe der IPoIB-Subnetze, die für BeeGFS-Clients und -Server verwendet werden, müssen Sie das erwartete Wachstum Ihres Compute/GPU-Clusters und BeeGFS-Dateisystems berücksichtigen. Wenn Sie von den empfohlenen IP-Bereichen abweichen müssen, beachten Sie, dass jede direkte Verbindung in einem einzelnen Baustein ein eigenes Subnetz hat und es keine Überschneidung mit Subnetzen gibt, die für die Client-Server-Konnektivität verwendet werden.

Direkte Verbindungen

Datei- und Block-Nodes innerhalb jedes Bausteins verwenden für ihre direkten Verbindungen immer die IPs in der folgenden Tabelle.

> Dieses Adressprogramm entspricht der folgenden Regel: Das dritte Oktett ist immer ungerade oder gerade, was davon abhängt, ob der Datei-Node ungerade oder gerade ist.

Datei-Node	IB-Port	IP-Adresse	Block-Node	IB-Port	Physische IP-Adresse	Virtuelle IP
ODD (h1)	i1a	192.168.1.10	Ungerade (c1)	2 a	192.168.1.100	192.168.1.101
ODD (h1)	i2a	192.168.3.10	Ungerade (c1)	2 a	192.168.3.100	192.168.3.101

Datei-Node	IB-Port	IP-Adresse	Block-Node	IB-Port	Physische IP-Adresse	Virtuelle IP
ODD (h1)	i3a	192.168.5.10	Gleichmäßig (c2)	2 a	192.168.5.100	192.168.5.101
ODD (h1)	I4a	192.168.7.10	Gleichmäßig (c2)	2 a	192.168.7.100	192.168.7.101
Gleichmäßig (h2)	i1a	192.168.2.10	Ungerade (c1)	2b	192.168.2.100	192.168.2.101
Gleichmäßig (h2)	i2a	192.168.4.10	Ungerade (c1)	2b	192.168.4.100	192.168.4.101
Gleichmäßig (h2)	i3a	192.168.6.10	Gleichmäßig (c2)	2b	192.168.6.100	192.168.6.101
Gleichmäßig (h2)	I4a	192.168.8.10	Gleichmäßig (c2)	2b	192.168.8.100	192.168.8.101

BeeGFS-Client-Server-IPoIB-Adressierschema (zwei Subnetze)

Damit BeeGFS-Clients zwei InfiniBand-Ports verwenden können, sind zwei IPoIB-Subnetze erforderlich, wobei die Hälfte der BeeGFS-Serverdienste, die mit einer bevorzugten IP-Adresse in jedem Subnetz konfiguriert sind, um sicherzustellen, dass Clients zwei InfiniBand-Ports verwenden, um die Redundanz und den möglichen Durchsatz zum Dateisystem zu maximieren.

Auf jedem Datei-Node werden mehrere BeeGFS-Serverservices ausgeführt (Management, Metadaten oder Storage). Damit jeder Service unabhängig vom anderen Datei-Node ein Failover durchführen kann, verfügt jeder über eindeutige IP-Adressen, die zwischen beiden Nodes schweben können (auch als logische Schnittstelle oder LIF bezeichnet).

Diese Bereitstellung setzt zwar nicht zwingend voraus, dass für diese Verbindungen folgende IPoIB-Subnetzbereiche verwendet werden und definiert ein Standard-Adressierungsschema, das folgende Regeln anwendet:

- Das zweite Oktett ist immer ungerade oder sogar, basierend darauf, ob der InfiniBand-Port des Datei-Nodes ungerade oder sogar ungerade ist.
- BeeGFS Cluster-IPs sind immer `xxx. 127.100.yyy` Oder `xxx.128.100.yyy`.

> Zusätzlich zur Schnittstelle, die für die bandinterne Betriebssystemverwaltung verwendet wird, können zusätzliche Schnittstellen von Corosync für Cluster Heart-Schläge und Synchronisation verwendet werden. So wird sichergestellt, dass der Verlust einer einzelnen Schnittstelle das gesamte Cluster nicht in den Fall bringt.

- Der BeeGFS Management Service ist immer im Betrieb `xxx.yyy.101.0` Oder `xxx.yyy.102.0`.
- BeeGFS Metadatendienste sind immer dabei `xxx.yyy.101.zzz` Oder `xxx.yyy.102.zzz`.
- BeeGFS Storage Services befinden sich immer im `xxx.yyy.103.zzz` Oder `xxx.yyy.103.zzz`.
- Adressen im Bereich `100.xxx.1.1` Bis `100.xxx.99.255` Sind für Kunden reserviert.

Subnetz A: 100.127.0.0/16

In der folgenden Tabelle ist der Bereich für Subnetz A angegeben: 100.127.0.0/16.

Zweck	InfiniBand-Port	IP-Adresse oder Bereich
BeeGFS Cluster-IP	i1b	100.127.100.1 - 100.127.100.255
BeeGFS Management	i1b	100.127.101.0
BeeGFS-Metadaten	i1b oder i3b	100.127.101.1 - 100.127.101.255
BeeGFS-Speicherung	i1b oder i3b	100.127.103.1 - 100.127.103.255
BeeGFS-Clients	(Je nach Kunde)	100.127.1.1 - 100.127.99.255

Subnetz B: 100.128.0.0/16

In der folgenden Tabelle ist der Bereich für Subnetz B angegeben: 100.128.0.0/16.

Zweck	InfiniBand-Port	IP-Adresse oder Bereich
BeeGFS Cluster-IP	I4b	100.128.100.1 - 100.128.100.255
BeeGFS Management	i2b	100.128.102.0
BeeGFS-Metadaten	i2b oder i4b	100.128.102.1 - 100.128.102.255
BeeGFS-Speicherung	i2b oder i4b	100.128.104.1 - 100.128.104.255
BeeGFS-Clients	(Je nach Kunde)	100.128.1.1 - 100.128.99.255

> In dieser NetApp Verified Architecture werden nicht alle IPs in den oben genannten Bereichen verwendet. Sie zeigen, wie IP-Adressen vorzugewiesen werden können, um eine einfache Erweiterung des Dateisystems mit einem konsistenten IP-Adressierungschema zu ermöglichen. In diesem Schema entsprechen BeeGFS-Datei-Knoten und Service-IDs dem vierten Oktett eines bekannten IP-Bereichs. Das Filesystem konnte bei Bedarf auf jeden Fall über 255 Nodes oder Services skaliert werden.

Implementierung von Hardware

Jeder Baustein besteht aus zwei validierten x86-Datei-Nodes, die mithilfe von HDR-Kabeln (200 GB) direkt mit zwei Block-Nodes verbunden sind.

> Da jeder Baustein zwei BeeGFS-Datei-Nodes enthält, sind mindestens zwei Bausteine erforderlich, um im Failover-Cluster ein Quorum einzurichten. Obwohl ein Cluster mit zwei Nodes konfiguriert werden kann, bestehen Einschränkungen bei dieser Konfiguration, die verhindern können, dass in einigen Szenarien ein erfolgreicher Failover auftritt. Wenn ein Cluster mit zwei Nodes erforderlich ist, kann ein drittes Gerät als Tiebreaker ebenfalls integriert werden, obwohl dieses Verfahren nicht abgedeckt ist.

Sofern nicht anders angegeben, bestehen die folgenden Schritte für jeden Baustein im Cluster identisch, unabhängig davon, ob er zum Ausführen von BeeGFS Metadaten und Storage-Services oder Storage-Services nur verwendet wird.

Schritte

1. Konfigurieren Sie jeden BeeGFS-Dateiknoten mit vier PCIe 4.0 ConnectX-6 Host Channel-Adaptern (HCAs) im InfiniBand-Modus und installieren Sie diese in den PCIe-Steckplätzen 2, 3, 5 und 6.
2. Konfigurieren Sie jeden BeeGFS-Block-Node mit einer 200-GB-Host-Schnittstellenkarte (HIC) mit zwei Ports, und installieren Sie die HIC in jedem ihrer beiden Storage Controller.

Stellen Sie die Bausteine so ein, dass die beiden BeeGFS-Datei-Nodes über den BeeGFS-Block-Nodes liegen. Die folgende Abbildung zeigt die richtige Hardwarekonfiguration für den BeeGFS-Baustein (Rückansicht).

[]

> Die Konfiguration der Stromversorgung für Produktionsanwendungsfälle sollte in der Regel redundante Netzteile verwenden.

3. Installieren Sie bei Bedarf die Laufwerke in jedem BeeGFS-Block-Knoten.
   a. Wenn der Baustein zur Ausführung von BeeGFS-Metadaten und Speicherdiensten verwendet wird und kleinere Laufwerke für Metadaten-Volumes verwendet werden, vergewissern Sie sich, dass diese in den äußeren Laufwerksschächten gefüllt sind, wie in der Abbildung unten gezeigt.
   b. Wenn ein Laufwerkgehäuse nicht vollständig bestückt ist, stellen Sie bei allen Bausteinkonfigurationen sicher, dass eine gleiche Anzahl an Laufwerken in den Steckplätzen 0–11 und 12–23 gefüllt ist, um eine optimale Performance zu erzielen.

      []

4. Um die Datei- und Block-Nodes zu verkabeln, verwenden Sie 1 m InfiniBand HDR 200 GB Direct Attach Kupferkabel, sodass sie mit der in der Abbildung unten dargestellten Topologie übereinstimmen.

   []

> Die Nodes sind über mehrere Bausteine hinweg nie direkt miteinander verbunden. Jeder Baustein sollte als eigenständige Einheit behandelt werden und alle Kommunikation zwischen Bausteinen erfolgt über Netzwerk-Switches.

5. Verwenden Sie 2 m (oder die entsprechende Länge) InfiniBand HDR 200 GB Direct-Attach-Kupferkabel, um die übrigen InfiniBand-Ports auf jedem Dateiknoten mit den InfiniBand-Switches zu verkabeln, die für das Speichernetzwerk verwendet werden sollen.

   Wenn redundante InfiniBand-Switches verwendet werden, verkabeln Sie die in der folgenden Abbildung grün hervorgehobenen Ports mit verschiedenen Switches.

   []

6. Montieren Sie bei Bedarf weitere Bausteine gemäß den gleichen Verkabelungsrichtlinien.

> Die Gesamtzahl der Bausteine, die in einem einzigen Rack implementiert werden können, hängt von der verfügbaren Stromversorgung und Kühlung an jedem Standort ab.

Implementierung von Software

Einrichten von Datei-Nodes und Block-Nodes

Während die meisten Software-Konfigurationsaufgaben mithilfe der von NetApp zur Verfügung gestellten Ansible Sammlungen automatisiert werden, müssen Sie das Netzwerk auf dem Baseboard Management Controller (BMC) jedes Servers konfigurieren und den Management-Port auf jedem Controller konfigurieren.

Richten Sie die Datei-Nodes ein

1. Konfigurieren Sie das Netzwerk auf dem Baseboard Management Controller (BMC) jedes Servers.

   Informationen zur Konfiguration von Netzwerken für die validierten Lenovo SR665-Dateiknoten finden Sie im "Lenovo ThinkSystem Dokumentation".

   > Ein Baseboard Management Controller (BMC), der manchmal als Service-Prozessor bezeichnet wird, ist der generische Name für die Out-of-Band-Management-Funktion, die in verschiedenen Server-Plattformen integriert ist, die Remote-Zugriff bieten können, selbst wenn das Betriebssystem nicht installiert ist oder nicht zugänglich ist. Anbieter vermarkten diese Funktionalität in der Regel mit ihrem eigenen Branding. Auf dem Lenovo SR665 wird beispielsweise der BMC als *Lenovo XClarity Controller (XCC)* bezeichnet.

2. Konfigurieren Sie die Systemeinstellungen für maximale Performance.

   Sie konfigurieren die Systemeinstellungen über das UEFI-Setup (früher BIOS) oder über die Redfish APIs, die von vielen BMCs bereitgestellt werden. Die Systemeinstellungen variieren je nach Servermodell, das als Dateiknoten verwendet wird.

   Informationen zur Konfiguration der Systemeinstellungen für die validierten Lenovo SR665-Dateiknoten finden Sie unter "Passen Sie die Systemeinstellungen für die Performance an".

3. Installieren Sie Red hat 8.4 und konfigurieren Sie den Hostnamen und den Netzwerkport, mit dem das Betriebssystem verwaltet wird, einschließlich SSH-Konnektivität über den Ansible-Steuerungs-Node.

   Konfigurieren Sie derzeit keine IPs auf einem der InfiniBand-Ports.

   > Die nachfolgenden Abschnitte gehen davon aus, dass die Hostnamen sequenziell nummeriert sind (z. B. h1-HN), und beziehen sich auf Aufgaben, die auf ungeraden oder gar nummerierten Hosts ausgeführt werden sollten.

4. Verwenden Sie den RedHat Subscription Manager, um das System zu registrieren und zu abonnieren, damit die erforderlichen Pakete aus den offiziellen Red hat Repositorys installiert werden können und um Updates auf die unterstützte Version von Red hat zu beschränken: `subscription-manager release --set=8.4`. Anweisungen hierzu finden Sie unter "Registrieren und Abonnieren eines RHEL Systems" Und "Einschränken von Aktualisierungen".
5. Aktivieren Sie das Red hat Repository mit den für hohe Verfügbarkeit erforderlichen Paketen.

   ```
   subscription-manager repo-override --repo=rhel-8-for-x86_64
   -highavailability-rpms --add=enabled:1
   ```

6. Aktualisieren Sie alle ConnectX-6 HCA-Firmware auf die in empfohlene Version "Technologieanforderungen erfüllt".

   Dieses Update kann durch Herunterladen und Ausführen einer Version des mlxup-Tools durchgeführt werden, das die empfohlene Firmware bündelt. Sie können dieses Tool von herunterladen "Mlxup - Dienstprogramm zum Aktualisieren und Abfragen" ("Benutzerhandbuch").

Richten Sie Block-Nodes ein

Richten Sie die EF600 Block-Nodes ein, indem Sie den Managementport pro Controller konfigurieren.

1. Konfigurieren Sie den Managementport an jedem EF600 Controller.

   Anweisungen zum Konfigurieren von Ports finden Sie im "E-Series Documentation Center".

2. Legen Sie optional den Speicher-Array-Namen für jedes System fest.

   Durch das Festlegen eines Namens kann es einfacher sein, in den nachfolgenden Abschnitten auf jedes System zu verweisen. Anweisungen zum Festlegen des Array-Namens finden Sie im "E-Series Documentation Center".

> Folgende Themen setzen voraus, dass die Namen der Speicherarrays nacheinander nummeriert sind (z. B. c1 - CN), und beziehen sich auf die Schritte, die auf ungeraden und nicht gerade nummerierten Systemen ausgeführt werden sollten.

Richten Sie einen Ansible-Steuerungsknoten ein

Zum Einrichten eines Ansible-Kontrollknotens müssen Sie eine virtuelle oder physische Maschine mit Netzwerkzugriff auf die Verwaltungsports aller Datei- und Block-Nodes identifizieren, die zum Konfigurieren der Lösung verwendet werden kann.

Die folgenden Schritte wurden auf CentOS 8.4 getestet. Weitere Schritte für Ihre bevorzugte Linux-Distribution finden Sie im "Ansible-Dokumentation".

1. Installieren Sie Python 3.9 und stellen Sie sicher, dass die richtige Version von vorhanden ist `pip` Installiert ist.

   ```
   sudo dnf install python3.9 -y
   sudo dnf install python39-pip
   sudo dnf install sshpass
   ```

2. Erstellen Sie symbolische Links, um sicherzustellen, dass die Python 3.9-Binärdatei immer verwendet wird `python3` Oder `python` Wird aufgerufen.

   ```
   sudo ln -sf /usr/bin/python3.9 /usr/bin/python3
   sudo ln -sf /usr/bin/python3 /usr/bin/python
   ```

3. Installieren Sie die Python-Pakete, die von den NetApp BeeGFS Collections benötigt werden.

   ```
   python3 -m pip install ansible cryptography netaddr
   ```

   > Um sicherzustellen, dass Sie eine unterstützte Version von Ansible und alle erforderlichen Python-Pakete installieren, lesen Sie die Readme-Datei der BeeGFS-Sammlung. Auch unterstützte Versionen sind in angegeben "Technische Anforderungen".

4. Vergewissern Sie sich, dass die korrekten Versionen von Ansible und Python installiert sind.

```
ansible --version
ansible [core 2.11.6]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-
packages/ansible
  ansible collection location =
/root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.9.2 (default, Mar 10 2021, 17:29:56) [GCC 8.4.1
20200928 (Red Hat 8.4.1-1)]
  jinja version = 3.0.2
  libyaml = True
```

5. Speichern der Ansible-Bestände, die zur Beschreibung der BeeGFS-Implementierung in Quellkontrollsystemen wie Git oder BitBucket verwendet werden, und installieren Sie Git, um mit diesen Systemen zu interagieren.

```
sudo dnf install git -y
```

6. Einrichten passwortloser SSH. Auf diese Weise kann Ansible am einfachsten über den Ansible-Steuerungsknoten auf die Remote BeeGFS-Datei-Nodes zugreifen.
   a. Generieren Sie auf dem Ansible-Kontroll-Node, falls erforderlich, mithilfe ein Paar öffentliche Schlüssel `ssh-keygen`
   b. Richten Sie mit jedem Dateiknoten passwortlose SSH ein `ssh-copy-id <ip_or_hostname>`

      Richten Sie bei den Blockknoten eine passwortlose SSH ein. Dies wird weder unterstützt noch erforderlich.

7. Verwenden Sie Ansible Galaxy, um die in aufgeführte BeeGFS-Version zu installieren "Technische Anforderungen".

   Diese Installation umfasst zusätzliche Ansible-Abhängigkeiten, z. B. die NetApp SANtricity Software und Host-Sammlungen.

```
ansible-galaxy collection install netapp_eseries.beegfs:==3.0.1
```

Erstellen des Ansible-Inventars

Um die Konfiguration für Datei- und Block-Nodes zu definieren, erstellen Sie einen Ansible-Bestand für das BeeGFS-Dateisystem, das bereitgestellt werden soll. Der Bestand umfasst Hosts, Gruppen und Variablen, die das gewünschte BeeGFS-

Dateisystem beschreiben.

Schritt 1: Konfiguration für alle Bausteine definieren

Legen Sie die Konfiguration fest, die für alle Bausteine gilt, unabhängig davon, welches Konfigurationsprofil Sie für sie einzeln anwenden können.

Bevor Sie beginnen

- Verwenden Sie ein Quellkontrollsystem wie BitBucket oder Git, um den Inhalt des Verzeichnisses mit dem Ansible-Inventar und den Playbook-Dateien zu speichern.
- Erstellen Sie ein `.gitignore` Datei, die die Dateien angibt, die Git ignorieren soll. Dadurch wird verhindert, dass große Dateien in Git gespeichert werden.

Schritte

1. Geben Sie auf dem Ansible-Steuerungsknoten ein Verzeichnis an, das Sie zum Speichern der Bestands- und Playbook-Dateien in Ansible verwenden möchten.

   Sofern nicht anders angegeben, werden alle in diesem Schritt erstellten Dateien und Verzeichnisse und die folgenden Schritte relativ zu diesem Verzeichnis erstellt.

2. Folgende Unterverzeichnisse erstellen:

   `host_vars`

   `group_vars`

   `packages`

Schritt: Konfiguration für einzelne Datei- und Block-Nodes definieren

Legen Sie die Konfiguration für einzelne Datei-Nodes und einzelne Baustein-Nodes fest.

1. Unter `host_vars/`Erstellen Sie für jeden BeeGFS-Dateiknoten eine Datei mit dem Namen `<HOSTNAME>.yml` Mit dem folgenden Inhalt, besondere Aufmerksamkeit auf die Notizen über den Inhalt für BeeGFS Cluster-IPs und Host-Namen enden in ungerade oder gerade Zahlen.

   Zunächst stimmen die Schnittstellennamen der Dateiknoten mit dem überein, was hier aufgeführt ist (z. B. ib0 oder ibs1f0). Diese benutzerdefinierten Namen werden in konfiguriert die für alle Datei-Knoten gelten soll.

```
ansible_host: "<MANAGEMENT_IP>"
eseries_ipoib_interfaces:  # Used to configure BeeGFS cluster IP
addresses.
  - name: i1b
    address: 100.127.100. <NUMBER_FROM_HOSTNAME>/16
  - name: i4b
    address: 100.128.100. <NUMBER_FROM_HOSTNAME>/16
beegfs_ha_cluster_node_ips:
  - <MANAGEMENT_IP>
  - <i1b_BEEGFS_CLUSTER_IP>
  - <i4b_BEEGFS_CLUSTER_IP>
# NVMe over InfiniBand storage communication protocol information
# For odd numbered file nodes (i.e., h01, h03, ..):
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.1.10/24
    configure: true
  - name: i2a
    address: 192.168.3.10/24
    configure: true
  - name: i3a
    address: 192.168.5.10/24
    configure: true
  - name: i4a
    address: 192.168.7.10/24
    configure: true
# For even numbered file nodes (i.e., h02, h04, ..):
# NVMe over InfiniBand storage communication protocol information
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.2.10/24
    configure: true
  - name: i2a
    address: 192.168.4.10/24
    configure: true
  - name: i3a
    address: 192.168.6.10/24
    configure: true
  - name: i4a
    address: 192.168.8.10/24
    configure: true
```

> Wenn Sie bereits das BeeGFS-Cluster implementiert haben, müssen Sie das Cluster beenden, bevor Sie statisch konfigurierte IP-Adressen hinzufügen oder ändern, einschließlich Cluster-IPs und IPs für NVMe/IB. Dies ist erforderlich, damit diese Änderungen ordnungsgemäß wirksam werden und Cluster-Vorgänge nicht unterbrechen.

2. Unter `host_vars/`Erstellen Sie für jeden BeeGFS-Block-Knoten eine Datei mit dem Namen `<HOSTNAME>.yml` Und geben Sie den folgenden Inhalt ein.

   Achten Sie besonders auf die Hinweise zum Inhalt, die für Speicher-Array-Namen ausgefüllt werden müssen, die mit ungeraden oder geraden Zahlen enden.

   Erstellen Sie für jeden Block-Node eine Datei, und geben Sie den an `<MANAGEMENT_IP>` Für einen der beiden Controller (normalerweise A).

```
eseries_system_name: <STORAGE_ARRAY_NAME>
eseries_system_api_url: https://<MANAGEMENT_IP>:8443/devmgr/v2/
eseries_initiator_protocol: nvme_ib
# For odd numbered block nodes (i.e., a01, a03, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101
    - 192.168.2.101
    - 192.168.1.100
    - 192.168.2.100
  controller_b:
    - 192.168.3.101
    - 192.168.4.101
    - 192.168.3.100
    - 192.168.4.100
# For even numbered block nodes (i.e., a02, a04, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.5.101
    - 192.168.6.101
    - 192.168.5.100
    - 192.168.6.100
  controller_b:
    - 192.168.7.101
    - 192.168.8.101
    - 192.168.7.100
    - 192.168.8.100
```

Schritt 3: Definieren Sie die Konfiguration, die für alle Datei- und Block-Nodes gelten soll

Unter können Sie die gemeinsame Konfiguration für eine Gruppe von Hosts definieren `group_vars` In einem Dateinamen, der der Gruppe entspricht. Dadurch wird verhindert, dass eine gemeinsame Konfiguration an mehreren Orten wiederholt wird.

Über diese Aufgabe

Hosts können sich in mehr als einer Gruppe befinden. Ansible zur Laufzeit wählt Ansible aus, welche Variablen auf Basis seiner variablen Rangfolge für einen bestimmten Host gelten. (Weitere Informationen zu diesen Regeln finden Sie in der Ansible-Dokumentation für "Variablen verwenden".)

Host-zu-Gruppe-Zuweisungen werden in der tatsächlichen Ansible-Bestandsdatei definiert, die gegen Ende dieses Vorgangs erstellt wird.

Schritt

In Ansible können alle Konfigurationen, die auf alle Hosts angewendet werden sollen, in einer Gruppe mit dem Namen definiert werden `All`. Erstellen Sie die Datei `group_vars/all.yml` Mit folgenden Inhalten:

```
ansible_python_interpreter: /usr/bin/python3
beegfs_ha_ntp_server_pools:  # Modify the NTP server addressess if
desired.
  - "pool 0.pool.ntp.org iburst maxsources 3"
  - "pool 1.pool.ntp.org iburst maxsources 3"
```

Schritt 4: Definieren Sie die Konfiguration, die für alle Datei-Knoten gelten soll

Die gemeinsame Konfiguration für Dateiknoten ist in einer Gruppe mit dem Namen definiert `ha_cluster`. In den Schritten in diesem Abschnitt wird die Konfiguration erstellt, die in der enthalten sein sollte `group_vars/ha_cluster.yml` Datei:

Schritte

1. Legen Sie oben in der Datei die Standardeinstellungen fest, einschließlich des Kennworts, das als verwendet werden soll `sudo` Benutzer auf den Datei-Nodes.

   ```
   ### ha_cluster Ansible group inventory file.
   # Place all default/common variables for BeeGFS HA cluster resources
   below.
   ### Cluster node defaults
   ansible_ssh_user: root
   ansible_become_password: <PASSWORD>
   eseries_ipoib_default_hook_templates:
     - 99-multihoming.j2 # This is required when configuring additional
   static IPs (for example cluster IPs) when multiple IB ports are in the
   same IPoIB subnet.
   # If the following options are specified, then Ansible will
   automatically reboot nodes when necessary for changes to take effect:
   eseries_common_allow_host_reboot: true
   eseries_common_reboot_test_command: "systemctl --state=active,exited |
   grep eseries_nvme_ib.service"
   ```

> Speichern Sie Passwörter insbesondere für Produktionsumgebungen nicht im Klartext. Verwenden Sie stattdessen den Ansible Vault (siehe "Verschlüsseln von Inhalten mit Ansible Vault") Oder der `--ask-become-pass` Option beim Ausführen des Playbooks. Wenn der `ansible_ssh_user` Ist bereits `root`, Dann können Sie optional die weglassen `ansible_become_password`.

2. Konfigurieren Sie optional einen Namen für den Hochverfügbarkeits-Cluster und geben Sie einen Benutzer für die Cluster-interne Kommunikation an.

   Wenn Sie das private IP-Adressschema ändern, müssen Sie auch die Standardeinstellung aktualisieren `beegfs_ha_mgmtd_floating_ip`. Dies muss mit dem übereinstimmen, was Sie später für die BeeGFS Management Ressourcengruppe konfigurieren.

   Geben Sie eine oder mehrere E-Mails an, die Warnmeldungen für Cluster-Ereignisse mit empfangen sollen `beegfs_ha_alert_email_list`.

```
### Cluster information
beegfs_ha_firewall_configure: True
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
# The following variables should be adjusted depending on the desired
configuration:
beegfs_ha_cluster_name: hacluster                  # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster              # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword             # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
beegfs_ha_mgmtd_floating_ip: 100.127.101.0         # BeeGFS management
service IP address.
# Email Alerts Configuration
beegfs_ha_enable_alerts: True
beegfs_ha_alert_email_list: ["email@example.com"]  # E-mail recipient
list for notifications when BeeGFS HA resources change or fail.  Often a
distribution list for the team responsible for managing the cluster.
beegfs_ha_alert_conf_ha_group_options:
      mydomain: “example.com”
# The mydomain parameter specifies the local internet domain name. This
is optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com).
# Adjusting the following parameters is optional:
beegfs_ha_alert_timestamp_format: "%Y-%m-%d %H:%M:%S.%N" #%H:%M:%S.%N
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

> Während scheinbar redundant, `beegfs_ha_mgmtd_floating_ip` Ist wichtig, wenn Sie das BeeGFS-Dateisystem über einen einzelnen HA-Cluster hinaus skalieren. Nachfolgende HA-Cluster werden ohne zusätzlichen BeeGFS-Managementservice bereitgestellt und Punkt am Managementservice des ersten Clusters.

3. Konfigurieren Sie einen Fechtagenten. (Weitere Informationen finden Sie unter "Konfigurieren Sie Fechten in einem Red hat High Availability Cluster".) Die folgende Ausgabe zeigt Beispiele für die Konfiguration von gemeinsamen Fechten-Agenten. Wählen Sie eine dieser Optionen.

   Beachten Sie bei diesem Schritt Folgendes:

   - Standardmäßig ist Fechten aktiviert, Sie müssen jedoch einen Fechten_Agent_ konfigurieren.

- Der `<HOSTNAME>` Angegeben in `pcmk_host_map` Oder `pcmk_host_list` Der Hostname in der Ansible-Bestandsaufnahme entspricht.
- Das BeeGFS-Cluster ohne Fencing wird insbesondere in der Produktion nicht unterstützt. Dies soll weitgehend sicherstellen, wenn BeeGFS-Services, einschließlich aller Ressourcenabhängigkeiten wie Blockgeräte, Failover aufgrund eines Problems durchführen, es besteht keine Möglichkeit des gleichzeitigen Zugriffs durch mehrere Nodes, die zu einer Beschädigung des Filesystems oder anderen unerwünschten oder unerwarteten Verhalten führen. Wenn das Fechten deaktiviert werden muss, lesen Sie die allgemeinen Hinweise in der BeeGFS HA-Rolle „erste Schritte“-Anleitung und „Set“ `beegfs_ha_cluster_crm_config_options["stonith-enabled"]` Mit FALSE innen `ha_cluster.yml`.
- Es sind mehrere Fechtgeräte auf Node-Ebene verfügbar, und die BeeGFS HA-Rolle kann jeden Fechtagenten konfigurieren, der im Red hat HA Package Repository verfügbar ist. Wenn möglich, verwenden Sie einen Zaunsagenten, der über die unterbrechungsfreie Stromversorgung (USV) oder die Rack-Stromverteilereinheit (rPDU) arbeitet. Da einige Fechten-Agenten wie der Baseboard-Management-Controller (BMC) oder andere Lights-Out-Geräte, die in den Server integriert sind, möglicherweise nicht auf die Zaunanforderung unter bestimmten Ausfallszenarien reagieren.

```
### Fencing configuration:
# OPTION 1: To enable fencing using APC Power Distribution Units
(PDUs):
beegfs_ha_fencing_agents:
 fence_apc:
   - ipaddr: <PDU_IP_ADDRESS>
     login: <PDU_USERNAME>
     passwd: <PDU_PASSWORD>
     pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>"
# OPTION 2: To enable fencing using the Redfish APIs provided by the
Lenovo XCC (and other BMCs):
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify “1”.
beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
# For details on configuring other fencing agents see
https://access.redhat.com/documentation/en-
us/red_hat_enterprise_linux/8/html/configuring_and_managing_high_avai
lability_clusters/assembly_configuring-fencing-configuring-and-
managing-high-availability-clusters.
```

4. Aktivieren Sie die empfohlene Performance-Optimierung im Linux-Betriebssystem.

   Viele Benutzer finden die Standardeinstellungen für die Performance-Parameter zwar im Allgemeinen gut, Sie können jedoch optional die Standardeinstellungen für einen bestimmten Workload ändern. Daher sind diese Empfehlungen in die BeeGFS-Rolle enthalten, jedoch sind sie nicht standardmäßig aktiviert, um sicherzustellen, dass Benutzer die auf ihr Dateisystem angewendete Einstellung kennen.

   Um das Performance-Tuning zu aktivieren, geben Sie Folgendes an:

   ```
   ### Performance Configuration:
   beegfs_ha_enable_performance_tuning: True
   ```

5. (Optional) Sie können die Leistungsparameter im Linux-Betriebssystem nach Bedarf anpassen.

   Eine umfassende Liste der verfügbaren Tuning-Parameter, die Sie anpassen können, finden Sie im

Abschnitt Performance Tuning Defaults der BeeGFS HA-Rolle in "E-Series BeeGFS GitHub-Website". Die Standardwerte können für alle Nodes im Cluster in dieser Datei oder im außer Kraft gesetzt werden `host_vars` Datei für einen einzelnen Knoten.

6. Um eine vollständige 200-GB-/HDR-Konnektivität zwischen Block- und Datei-Nodes zu ermöglichen, verwenden Sie das OpenSubnetz Manager-Paket (OpenSM) aus der Mellanox Open Fabrics Enterprise Distribution (MLNX_OFED). (Posteingang `opensm` Das Paket unterstützt nicht die erforderlichen Virtualisierungsfunktionen.) Obwohl die Implementierung mit Ansible unterstützt wird, müssen Sie die gewünschten Pakete zuerst auf den Ansible-Kontroll-Node herunterladen, der zum Ausführen der BeeGFS-Rolle verwendet wird.

   a. Wird Verwendet `curl` Oder Ihr gewünschtes Tool, laden Sie die Pakete für die Version von OpenSM im Abschnitt Technologieanforderungen aufgeführt von Mellanox Website auf die `packages/` Verzeichnis. Beispiel:

```
curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm

curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
```

   b. Füllen Sie die folgenden Parameter in aus `group_vars/ha_cluster.yml` (Passen Sie Pakete nach Bedarf an):

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
eseries_ib_opensm_options:
  virt_enabled: "2"
```

7. Konfigurieren Sie die `udev` Regel zur Sicherstellung einer konsistenten Zuordnung von logischen InfiniBand-Port-IDs zu zugrunde liegenden PCIe-Geräten.

   Der `udev` Die Regel muss für die PCIe-Topologie jeder Serverplattform, die als BeeGFS-Datei-Node verwendet wird, eindeutig sein.

   Für verifizierte Dateiknoten folgende Werte verwenden:

```
### Ensure Consistent Logical IB Port Numbering
# OPTION 1: Lenovo SR665 PCIe address-to-logical IB port mapping:
eseries_ipoib_udev_rules:
  "0000:41:00.0": i1a
  "0000:41:00.1": i1b
  "0000:01:00.0": i2a
  "0000:01:00.1": i2b
  "0000:a1:00.0": i3a
  "0000:a1:00.1": i3b
  "0000:81:00.0": i4a
  "0000:81:00.1": i4b

# Note: At this time no other x86 servers have been qualified.
Configuration for future qualified file nodes will be added here.
```

8. (Optional) Aktualisieren des Metadaten-Zielauswahlalgorithmus.

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  tuneTargetChooser: randomrobin
```

> Während der Verifizierungstests `randomrobin` Wurde in der Regel verwendet, um sicherzustellen, dass Testdateien während des Performance-Benchmarking gleichmäßig auf alle BeeGFS-Speicherziele verteilt wurden (weitere Informationen zu Benchmarking finden Sie auf der BeeGFS-Website für "Benchmarking eines BeeGFS-Systems"). Bei der realen Welt könnte dies dazu führen, dass sich die niedrigeren nummerierten Ziele schneller füllen als die höher nummerierten Ziele. Auslassung `randomrobin` Und nur mit dem Standard `randomized` Der Wert zeigt sich, dass er eine gute Leistung bietet und gleichzeitig alle verfügbaren Ziele nutzt.

Schritt 5: Definieren Sie die Konfiguration für den gemeinsamen Block-Node

Die gemeinsame Konfiguration für Block-Knoten wird in einer Gruppe mit dem Namen definiert `eseries_storage_systems`. In den Schritten in diesem Abschnitt wird die Konfiguration erstellt, die in der enthalten sein sollte `group_vars/ eseries_storage_systems.yml` Datei:

Schritte

1. Setzen Sie die Ansible-Verbindung auf Local, geben Sie das Systemkennwort ein und geben Sie an, ob SSL-Zertifikate verifiziert werden sollen. (Normalerweise verwendet Ansible SSH für die Verbindung zu gemanagten Hosts. Bei Storage-Systemen der NetApp E-Series, die als Block-Nodes verwendet werden, verwenden die Module JEDOCH die REST-API für die Kommunikation.) Fügen Sie oben in der Datei Folgendes hinzu:

```
### eseries_storage_systems Ansible group inventory file.
# Place all default/common variables for NetApp E-Series Storage Systems
here:
ansible_connection: local
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> Es wird nicht empfohlen, Kennwörter im Klartext zu verwenden. Verwenden Sie einen Ansible-Vault, oder stellen Sie die bereit `eseries_system_password` Bei Ausführung von Ansible mit `--extra-vars`.

2. Installieren Sie die für Block-Nodes in aufgeführten Versionen, um eine optimale Performance zu gewährleisten "Technische Anforderungen".

   Laden Sie die entsprechenden Dateien aus dem herunter "NetApp Support Website". Sie können sie entweder manuell aktualisieren oder sie in das einbeziehen `packages/` Verzeichnis des Ansible-Steuerungsknotens, und füllen Sie dann die folgenden Parameter in aus `eseries_storage_systems.yml` So führen Sie ein Upgrade mit Ansible durch:

```
# Firmware, NVSRAM, and Drive Firmware (modify the filenames as needed):
eseries_firmware_firmware: "packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/N6000-872834-D06.dlp"
```

3. Laden Sie die neueste Laufwerk-Firmware herunter, die für die Laufwerke verfügbar ist, die in Ihren Block-Nodes installiert sind, von "NetApp Support Website". Sie können sie entweder manuell aktualisieren oder sie in das einbeziehen `packages/` Verzeichnis des Ansible-Steuerungsknotens, und füllen Sie dann die folgenden Parameter in aus `eseries_storage_systems.yml` So führen Sie ein Upgrade mit Ansible durch:

```
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
eseries_drive_firmware_upgrade_drives_online: true
```

> Einstellung `eseries_drive_firmware_upgrade_drives_online` Bis `false` Beschleunigt das Upgrade, sollte aber erst nach dem Einsatz von BeeGFS durchgeführt werden. Der Grund dafür ist, dass bei dieser Einstellung sämtliche I/O-Vorgänge auf den Laufwerken vor dem Upgrade angehalten werden müssen, um Applikationsfehler zu vermeiden. Obwohl ein Online-Laufwerk-Firmware-Upgrade vor der Konfiguration von Volumes noch schnell durchgeführt wird, empfehlen wir Ihnen, diesen Wert immer auf zu setzen `true` Um später Probleme zu vermeiden.

4. Nehmen Sie zur Optimierung der Leistung folgende Änderungen an der globalen Konfiguration vor:

```
# Global Configuration Defaults
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required.
```

5. Um eine optimale Bereitstellung und ein optimales Verhalten von Volumes zu gewährleisten, geben Sie folgende Parameter an:

```
# Storage Provisioning Defaults
eseries_volume_size_unit: pct
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99:6,
99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

> Der für angegebene Wert `eseries_storage_pool_usable_drives` Gibt einen spezifischen Block-Node der NetApp EF600 an und steuert die Reihenfolge, in der Laufwerke neuen Volume-Gruppen zugewiesen werden. Durch diese Bestellung wird sichergestellt, dass der I/O zu jeder Gruppe gleichmäßig über die Kanäle des Backend-Laufwerks verteilt wird.

Definieren Sie den Ansible-Bestand für BeeGFS-Bausteine

Definieren Sie nach der Definition der allgemeinen Ansible-Bestandsstruktur die Konfiguration für jeden Baustein im BeeGFS-Dateisystem.

Diese Implementierungsanleitungen zeigen, wie Sie ein Filesystem implementieren, das aus einem Grundbaustein besteht, einschließlich Management-, Metadaten- und Storage-Services, einem zweiten Baustein mit Metadaten und Storage-Services und einem dritten Baustein nur für Storage.

Diese Schritte sollen den gesamten Bereich typischer Konfigurationsprofile anzeigen, mit denen Sie NetApp BeeGFS-Bausteine konfigurieren können, um die Anforderungen des gesamten BeeGFS-Dateisystems zu erfüllen.

> Passen Sie in diesen und nachfolgenden Abschnitten nach Bedarf an, um den Bestand zu erstellen, der das BeeGFS-Dateisystem darstellt, das Sie bereitstellen möchten. Verwenden Sie insbesondere Ansible-Hostnamen, die jeden Block- oder Datei-Node darstellen, und das gewünschte IP-Adressschema für das Storage-Netzwerk, um sicherzustellen, dass es auf die Anzahl der BeeGFS-Datei-Nodes und -Clients skaliert werden kann.

Schritt: Die Ansible-Bestandsdatei erstellen

Schritte

1. Erstellen Sie eine neue `inventory.yml` Datei, und fügen Sie dann die folgenden Parameter ein, ersetzen Sie die Hosts unter `eseries_storage_systems` Nach Bedarf zur Darstellung der Block-Nodes in Ihrer Implementierung. Die Namen sollten dem Namen entsprechen, für den sie verwendet werden `host_vars/<FILENAME>.yml`.

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

   In den nachfolgenden Abschnitten werden unter weitere Ansible-Gruppen erstellt `ha_cluster` Die die BeeGFS-Dienste darstellen, die im Cluster ausgeführt werden sollen.

Schritt: Inventar für einen Management-, Metadaten- und Storage-Baustein konfigurieren

Der erste Baustein im Cluster- oder Basis-Baustein muss den BeeGFS-Managementservice sowie Metadaten- und Storage-Services umfassen:

Schritte

1. In `inventory.yml`, Befüllen Sie die folgenden Parameter unter `ha_cluster: children`:

```
      # ictad22h01/ictad22h02 HA Pair (mgmt/meta/storage building
block):
        mgmt:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_01:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_01:
```

```
          hosts:
            ictad22h01:
            ictad22h02:
        meta_02:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_02:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_03:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_03:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_04:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_04:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_05:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_05:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_06:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_06:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_07:
          hosts:
            ictad22h02:
```

```
            ictad22h01:
        stor_07:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_08:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

2. Erstellen Sie die Datei `group_vars/mgmt.yml` Und geben Sie Folgendes an:

```
# mgmt - BeeGFS HA Management Resource Group
# OPTIONAL: Override default BeeGFS management configuration:
# beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
#  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
floating_ips:
  - i1b: 100.127.101.0/16
  - i2b: 100.128.102.0/16
beegfs_service: management
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 1
            owning_controller: A
```

3. Unter `group_vars/`, Dateien für Ressourcengruppen erstellen `meta_01` Bis `meta_08` Verwenden Sie die folgende Vorlage und füllen Sie dann die Platzhalterwerte für jeden Service aus, indem Sie auf die folgende Tabelle verweisen:

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET> # Example: i1b:192.168.120.1/16
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 21.25 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

Die Volume-Größe wird als Prozentsatz des gesamten Storage-Pools angegeben (auch als Volume-Gruppe bezeichnet). NetApp empfiehlt, freie Kapazitäten in jedem Pool zu belassen, um Platz für die SSD-Überprovisionierung zu haben (weitere Informationen finden Sie unter "Einführung in das NetApp EF600 Array"). Der Storage-Pool, `beegfs_m1_m2_m5_m6`, Weist auch 1% der Kapazität des Pools für den Management-Service. Somit für Metadaten-Volumes im Storage-Pool `beegfs_m1_m2_m5_m6`, Wenn 1,92-TB- oder 3,84-TB-Laufwerke verwendet werden, setzen Sie diesen Wert auf `21.25`; Für 7,5-TB-Laufwerke setzen Sie diesen Wert auf `22.25`; Und für 15,3-TB-Laufwerke ist dieser Wert auf festgelegt `23.75`.

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
meta_01.yml	8015	i1b:100.127.101.1/16 i2b:100.128.102.1/16	0	Ictad22a01	Beegfs_m1_m2_m5_m6	A
meta_02.yml	8025	i2b:100.128.102.2/16 i1b:100.127.101.2/16	0	Ictad22a01	Beegfs_m1_m2_m5_m6	B
meta_03.yml	8035	i3b:100.127.101.3/16 i4b:100.128.102.3/16	1	Ictad22a02	Beegfs_m3_m4_m7_m8	A

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
meta_04.yml	8045	l4b:100.128.102.4/16 i3b:100.127.101.4/16	1	lctad22a02	Beegfs_m3_m4_m7_m8	B
meta_05.yml	8055	i1b:100.127.101.5/16 i2b:100.128.102.5/16	0	lctad22a01	Beegfs_m1_m2_m5_m6	A
meta_06.yml	8065	i2b:100.128.102.6/16 i1b:100.127.101.6/16	0	lctad22a01	Beegfs_m1_m2_m5_m6	B
meta_07.yml	8075	i3b:100.127.101.7/16 i4b:100.128.102.7/16	1	lctad22a02	Beegfs_m3_m4_m7_m8	A
meta_08.yml	8085	l4b:100.128.102.8/16 i3b:100.127.101.8/16	1	lctad22a02	Beegfs_m3_m4_m7_m8	B

4. Unter `group_vars/`, Dateien für Ressourcengruppen erstellen `stor_01` Bis `stor_08` Füllen Sie anschließend die Platzhalterwerte für jeden Service aus, indem Sie auf das Beispiel verweisen:

```
# stor_0X - BeeGFS HA Storage Resource
Groupbeegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!             owning_controller:
<OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

Informationen zur richtigen Größe finden Sie unter "Empfohlene Prozentsätze für die Überprovisionierung von Storage-Pools".

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
stor_01.yml	8013	i1b:100.127.103.1/16 i2b:100.128.104.1/16	0	lctad22a01	Beegfs_s1_s2	A
stor_02.yml	8023	i2b:100.128.104.2/16 i1b:100.127.103.2/16	0	lctad22a01	Beegfs_s1_s2	B
stor_03.yml	8033	i3b:100.127.103.3/16 i4b:100.128.104.3/16	1	lctad22a02	Beegfs_s3_s4	A
stor_04.yml	8043	l4b:100.128.104.4/16 i3b:100.127.103.4/16	1	lctad22a02	Beegfs_s3_s4	B

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
stor_05.yml	8053	i1b:100.127.103.5/16 i2b:100.128.104.5/16	0	Ictad22a01	Beegfs_s5_s6	A
stor_06.yml	8063	i2b:100.128.104.6/16 i1b:100.127.103.6/16	0	Ictad22a01	Beegfs_s5_s6	B
stor_07.yml	8073	i3b:100.127.103.7/16 i4b:100.128.104.7/16	1	Ictad22a02	Beegfs_s7_s8	A
stor_08.yml	8083	I4b:100.128.104.8/16 i3b:100.127.103.8/16	1	Ictad22a02	Beegfs_s7_s8	B

Schritt 3: Konfigurieren Sie den Bestand für einen Baustein Metadaten + Speicher

In diesen Schritten wird beschrieben, wie ein Ansible-Inventar für BeeGFS-Metadaten + Storage-Baustein eingerichtet wird.

Schritte

1. In `inventory.yml`, Befüllen Sie die folgenden Parameter unter der vorhandenen Konfiguration:

```
        meta_09:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_09:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_10:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_10:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_11:
          hosts:
            ictad22h03:
```

```
            ictad22h04:
        stor_11:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_12:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_12:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_13:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_13:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_14:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_14:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_15:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_15:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_16:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_16:
          hosts:
            ictad22h04:
            ictad22h03:
```

2. Unter `group_vars/`, Dateien für Ressourcengruppen erstellen `meta_09` Bis `meta_16` Füllen Sie anschließend die Platzhalterwerte für jeden Service aus, indem Sie auf das Beispiel verweisen:

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.5 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

Informationen zur richtigen Größe finden Sie unter "Empfohlene Prozentsätze für die Überprovisionierung von Storage-Pools".

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
meta_09.yml	8015	i1b:100.127.101.9/16 i2b:100.128.102.9/16	0	Ictad22a03	Beegfs_m9_m10_m13_m14	A
meta_10.yml	8025	i2b:100.128.102.10/16 i1b:100.127.101.10/16	0	Ictad22a03	Beegfs_m9_m10_m13_m14	B
meta_11.yml	8035	i3b:100.127.101.11/16 i4b:100.128.102.11/16	1	Ictad22a04	Beegfs_m11_m12_m15_m16	A
meta_12.yml	8045	I4b:100.128.102.12/16 i3b:100.127.101.12/16	1	Ictad22a04	Beegfs_m11_m12_m15_m16	B

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
meta_13.yml	8055	i1b:100.127.101.13/16 i2b:100.128.102.13/16	0	lctad22a03	Beegfs_m9_m10_m13_m14	A
meta_14.yml	8065	i2b:100.128.102.14/16 i1b:100.127.101.14/16	0	lctad22a03	Beegfs_m9_m10_m13_m14	B
meta_15.yml	8075	i3b:100.127.101.15/16 i4b:100.128.102.15/16	1	lctad22a04	Beegfs_m11_m12_m15_m16	A
meta_16.yml	8085	l4b:100.128.102.16/16 i3b:100.127.101.16/16	1	lctad22a04	Beegfs_m11_m12_m15_m16	B

3. Unter `group_vars/`, Dateien für Ressourcengruppen erstellen `stor_09` Bis `stor_16` Füllen Sie anschließend die Platzhalterwerte für jeden Service aus, indem Sie auf das Beispiel verweisen:

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

Informationen zur richtigen Größe finden Sie unter "Empfohlene Prozentsätze für die Überprovisionierung von Storage-Pools"..

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
stor_09.yml	8013	i1b:100.127.103.9/16 i2b:100.128.104.9/16	0	lctad22a03	Beegfs_s9_s10	A
stor_10.yml	8023	i2b:100.128.104.10/16 i1b:100.127.103.10/16	0	lctad22a03	Beegfs_s9_s10	B
stor_11.yml	8033	i3b:100.127.103.11/16 i4b:100.128.104.11/16	1	lctad22a04	Beegfs_s11_s12	A
stor_12.yml	8043	I4b:100.128.104.12/16 i3b:100.127.103.12/16	1	lctad22a04	Beegfs_s11_s12	B
stor_13.yml	8053	i1b:100.127.103.13/16 i2b:100.128.104.13/16	0	lctad22a03	Beegfs_s13_s14	A
stor_14.yml	8063	i2b:100.128.104.14/16 i1b:100.127.103.14/16	0	lctad22a03	Beegfs_s13_s14	B
stor_15.yml	8073	i3b:100.127.103.15/16 i4b:100.128.104.15/16	1	lctad22a04	Beegfs_s15_s16	A
stor_16.yml	8083	I4b:100.128.104.16/16 i3b:100.127.103.16/16	1	lctad22a04	Beegfs_s15_s16	B

Schritt 4: Konfigurieren Sie den Bestand für einen nur-Storage-Baustein

In diesen Schritten wird beschrieben, wie Sie einen Ansible-Bestand für einen einzigen BeeGFS-Baustein einrichten. Der Hauptunterschied zwischen der Konfiguration für Metadaten + Storage und einem rein Storage-basierten Baustein besteht darin, dass alle Metadaten-Ressourcengruppen und Änderungen nicht mehr berücksichtigt werden `criteria_drive_count` Von 10 bis 12 für jeden Speicherpool.

Schritte

1. In `inventory.yml`, Befüllen Sie die folgenden Parameter unter der vorhandenen Konfiguration:

```
    # ictad22h05/ictad22h06 HA Pair (storage only building block):
      stor_17:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_18:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_19:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_20:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_21:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_22:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_23:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_24:
        hosts:
          ictad22h06:
          ictad22h05:
```

2. Unter `group_vars/`, Dateien für Ressourcengruppen erstellen `stor_17` Bis `stor_24` Füllen Sie anschließend die Platzhalterwerte für jeden Service aus, indem Sie auf das Beispiel verweisen:

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 12
        common_volume_configuration:
          segment_size_kb: 512
        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50
            owning_controller: <OWNING CONTROLLER>
```

Informationen zur richtigen Größe finden Sie unter "Empfohlene Prozentsätze für die Überprovisionierung von Storage-Pools".

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
stor_17.yml	8013	i1b:100.127.103.17/16 i2b:100.128.104.17/16	0	lctad22a05	Beegfs_s17_s18	A
stor_18.yml	8023	i2b:100.128.104.18/16 i1b:100.127.103.18/16	0	lctad22a05	Beegfs_s17_s18	B
stor_19.yml	8033	i3b:100.127.103.19/16 i4b:100.128.104.19/16	1	lctad22a06	Beegfs_s19_s20	A
stor_20.yml	8043	l4b:100.128.104.20/16 i3b:100.127.103.20/16	1	lctad22a06	Beegfs_s19_s20	B

Dateiname	Port	Fließende IPs	NUMA-Zone	Block-Node	Storage-Pool	Controller, der die LUN besitzt
stor_21.yml	8053	i1b:100.127.103.21/16 i2b:100.128.104.21/16	0	lctad22a05	Beegfs_s21_s22	A
stor_22.yml	8063	i2b:100.128.104.22/16 i1b:100.127.103.22/16	0	lctad22a05	Beegfs_s21_s22	B
stor_23.yml	8073	i3b:100.127.103.23/16 i4b:100.128.104.23/16	1	lctad22a06	Beegfs_s23_s24	A
stor_24.yml	8083	I4b:100.128.104.24/16 i3b:100.127.103.24/16	1	lctad22a06	Beegfs_s23_s24	B

BeeGFS bereitstellen

Zur Implementierung und zum Management der Konfiguration werden ein oder mehrere Playbooks ausgeführt, die die Aufgaben enthalten, die Ansible ausführen muss, und das gesamte System in den gewünschten Zustand bringen.

Zwar können alle Aufgaben in einem einzigen Playbook enthalten sein, doch bei komplexen Systemen ist dies schnell und schwerfällig. Mit Ansible können Sie Rollen erstellen und verteilen, um wiederverwendbare Playbooks und verwandte Inhalte (z. B. Standardvariablen, Aufgaben und Handler) zu verpacken. Weitere Informationen finden Sie in der Ansible-Dokumentation für "Rollen".

Rollen werden häufig im Rahmen einer Ansible Sammlung mit zugehörigen Rollen und Modulen verteilt. Daher importieren diese Playbooks in erster Linie mehrere Rollen, die in den verschiedenen NetApp E-Series Ansible Sammlungen verteilt sind.

> Derzeit sind mindestens zwei Bausteine (vier Datei-Nodes) für die Bereitstellung von BeeGFS erforderlich, es sei denn, ein separates Quorum-Gerät ist als Tiebreaker konfiguriert, um Probleme beim Einrichten von Quorum mit einem Cluster mit zwei Nodes zu minimieren.

Schritte

1. Erstellen Sie eine neue `playbook.yml` Datei und schließen Sie Folgendes ein:

```
# BeeGFS HA (High Availability) cluster playbook.
- hosts: eseries_storage_systems
  gather_facts: false
  collections:
    - netapp_eseries.santricity
  tasks:
```

```
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version
        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'
        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true
        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
groups[beegfs_ha_ansible_storage_group]
    - name: Verify any provided tags are supported.
      fail:
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
```

```
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_2
```

> In diesem Playbook wird ein paar ausgeführt `pre_tasks` Überprüfen Sie, ob Python 3 auf den Datei-Nodes installiert ist, und überprüfen Sie, ob die angegebenen Ansible-Tags unterstützt werden.

2. Verwenden Sie die `ansible-playbook` Befehl mit den Inventar- und Playbook-Dateien, wenn Sie bereit sind BeeGFS zu implementieren.

   Die Bereitstellung wird komplett ausgeführt `pre_tasks`, Und dann zur Bestätigung des Benutzers aufgefordert, bevor mit der tatsächlichen BeeGFS-Bereitstellung.

   Führen Sie den folgenden Befehl aus, indem Sie die Anzahl der Gabeln nach Bedarf anpassen (siehe Hinweis unten):

```
ansible-playbook -i inventory.yml playbook.yml --forks 20
```

> Vor allem bei größeren Bereitstellungen überschreiben Sie die Standardanzahl der Gabeln (5) mit dem `forks` Der Parameter wird empfohlen, um die Anzahl der Hosts zu erhöhen, die Ansible parallel konfiguriert. (Weitere Informationen finden Sie unter "Ansible-Performance-Tuning" Und "Kontrolle der Playbook-Ausführung".) Die Einstellung für den maximalen Wert hängt von der verfügbaren Verarbeitungsleistung des Ansible-Steuerungsknotens ab. Das oben genannte Beispiel von 20 wurde auf einem virtuellen Ansible-Steuerungsknoten mit 4 CPUs (Intel® Xeon® Gold 6146 CPU @ 3,20 GHz) ausgeführt.

   Je nach Größe der Implementierung und Netzwerk-Performance zwischen dem Ansible Control Node und BeeGFS File- und Block-Nodes kann die Implementierungszeit variieren.

Konfigurieren Sie BeeGFS-Clients

Sie müssen den BeeGFS-Client auf allen Hosts installieren und konfigurieren, die Zugriff auf das BeeGFS-Dateisystem benötigen, z. B. Compute- oder GPU-Nodes. Für diese

Aufgabe können Sie Ansible und die BeeGFS-Sammlung verwenden.

Schritte

1. Richten Sie bei Bedarf über den Ansible-Steuerungsknoten passwortlose SSH für jeden Host ein, den Sie als BeeGFS-Clients konfigurieren möchten:

   ```
   ssh-copy-id <user>@<HOSTNAME_OR_IP>
   ```

2. Unter `host_vars/`Erstellen Sie für jeden BeeGFS-Client eine Datei mit dem Namen `<HOSTNAME>.yml` Füllen Sie den Platzhaltertext mit den korrekten Informationen für Ihre Umgebung aus:

   ```
   # BeeGFS Client
   ansible_host: <MANAGEMENT_IP>
   # OPTIONAL: If you want to use the NetApp E-Series Host Collection's
   IPoIB role to configure InfiniBand interfaces for clients to connect to
   BeeGFS file systems:
   eseries_ipoib_interfaces:
     - name: <INTERFACE>
       address: <IP>/<SUBNET_MASK> # Example: 100.127.1. 1/16
     - name: <INTERFACE>0
       address: <IP>/<SUBNET_MASK>
   ```

   > Derzeit müssen für jeden Client zwei InfiniBand-Schnittstellen konfiguriert werden, eine in jedem der beiden Storage-IPoIB-Subnetze. Wenn Sie die Beispielnetze und empfohlene Bereiche für jeden hier aufgeführten BeeGFS-Dienst verwenden, sollten Clients eine Schnittstelle im Bereich von haben `100.127.1. 0` Bis `100.127.99.255` Und das andere in `100.128.1. 0` Bis `100.128. 99.255`.

3. Erstellen Sie eine neue Datei `client_inventory.yml`, Und dann füllen Sie die folgenden Parameter an der Spitze:

   ```
   # BeeGFS client inventory.
   all:
     vars:
       ansible_ssh_user: <USER> # This is the user Ansible should use to
   connect to each client.
       ansible_become_password: <PASSWORD> # This is the password Ansible
   will use for privilege escalation, and requires the ansible_ssh_user be
   root, or have sudo privileges.
   The defaults set by the BeeGFS HA role are based on the testing
   performed as part of this NetApp Verified Architecture and differ from
   the typical BeeGFS client defaults.
   ```

> Speichern Sie Passwörter nicht im Klartext. Verwenden Sie stattdessen den Ansible Vault (siehe Ansible-Dokumentation für) "Verschlüsseln von Inhalten mit Ansible Vault") Oder verwenden Sie die `--ask-become-pass` Option beim Ausführen des Playbooks.

4. Im `client_inventory.yml` Datei, Listen Sie alle Hosts auf, die als BeeGFS-Clients unter dem konfiguriert werden sollen `beegfs_clients` Gruppe, und geben Sie dann alle zusätzlichen Konfigurationen an, die zum Erstellen des BeeGFS-Client-Kernelmoduls erforderlich sind.

```
  children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        ictad21h01:
        ictad21h02:
        ictad21h03:
        ictad21h04:
        ictad21h05:
        ictad21h06:
        ictad21h07:
        ictad21h08:
        ictad21h09:
        ictad21h10:
      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        beegfs_client_ofed_enable: True
        beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"
        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        eseries_ib_skip: False # Default value.
        beegfs_client_ofed_enable: False # Default value.
```

> Stellen Sie beim Verwenden der Mellanox OFED-Treiber sicher, dass dies der Fall ist `beegfs_client_ofed_include_path` Weist auf den richtigen „Header include path“ für Ihre Linux-Installation hin. Weitere Informationen finden Sie in der BeeGFS-Dokumentation für "RDMA-Unterstützung".

5. Im `client_inventory.yml` Datei, Listen Sie die BeeGFS-Dateisysteme auf, die am unteren Rand eines zuvor definierten gemountet werden sollen `vars`.

```
        beegfs_client_mounts:
          - sysMgmtdHost: 100.127.101.0 # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default "sync" mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
```

Der `beegfs_client_config` Stellt die Einstellungen dar, die getestet wurden. Lesen Sie die Dokumentation im `netapp_eseries.beegfs` Kollektion's `beegfs_client` Rolle für einen umfassenden Überblick über alle Optionen. Dies enthält Details zum Mounten mehrerer BeeGFS-Dateisysteme oder zum mehrere Male das gleiche BeeGFS-Dateisystem.

6. Erstellen Sie eine neue `client_playbook.yml` Datei, und füllen Sie dann die folgenden Parameter aus:

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

> Beim Importieren des weglassen `netapp_eseries.host` Sammlung und `ipoib` Rolle, wenn Sie bereits die erforderlichen IB/RDMA-Treiber und IP-Adressen auf den entsprechenden IPoIB-Schnittstellen installiert haben.

7. Führen Sie den folgenden Befehl aus, um den Client zu installieren und zu erstellen und BeeGFS zu mounten:

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

8. Bevor Sie das BeeGFS-Dateisystem in Produktion setzen, empfehlen wir * dringend*, sich bei allen Clients anzumelden und zu starten `beegfs-fsck --checkfs` Um sicherzustellen, dass alle Knoten erreichbar sind und keine Probleme gemeldet werden.

Skalierung auf mehr als fünf Bausteine

Pacemaker und Corosync können so konfiguriert werden, dass sie über fünf Bausteine (10 Datei-Knoten) hinausgehen. Allerdings gibt es Nachteile zu größeren Clustern, und schließlich Pacemaker und Corosync auferlegen ein Maximum von 32 Knoten.

NetApp hat nur BeeGFS HA Cluster für bis zu 10 Nodes getestet. Es wird nicht empfohlen, einzelne Cluster über dieses Limit hinaus zu skalieren. BeeGFS-Filesysteme müssen jedoch immer noch weit über 10 Nodes skalieren, und NetApp hat dies in der BeeGFS on NetApp Lösung berücksichtigt.

Durch die Implementierung mehrerer HA-Cluster mit einer Teilmenge der Bausteine in jedem Filesystem können Sie das gesamte BeeGFS Filesystem unabhängig von empfohlenen oder festen Grenzwerten für die zugrunde liegenden HA-Clustering-Mechanismen skalieren. Gehen Sie in diesem Szenario wie folgt vor:

- Erstellen Sie einen neuen Ansible-Bestand, der die zusätzlichen HA-Cluster darstellt, und geben Sie dann das Konfigurieren eines anderen Managementservices nicht ein. Zeigen Sie stattdessen auf `beegfs_ha_mgmtd_floating_ip` Variable in jedem zusätzlichen Cluster enthalten `ha_cluster.yml` An die IP für den ersten BeeGFS Management Service.

- Wenn Sie dem selben Dateisystem zusätzliche HA-Cluster hinzufügen, stellen Sie Folgendes sicher:
  - Die BeeGFS-Knoten-IDs sind eindeutig.
  - Die Dateinamen, die den einzelnen Diensten unter entsprechen `group_vars` Ist für alle Cluster eindeutig.
  - Die BeeGFS-Client- und Server-IP-Adressen sind für alle Cluster eindeutig.
  - Das erste HA-Cluster mit dem BeeGFS-Managementservice wird ausgeführt, bevor versucht wird, zusätzliche Cluster zu implementieren oder zu aktualisieren.
- Inventarisierung für jedes HA-Cluster getrennt in der eigenen Verzeichnisstruktur

  Wenn Sie versuchen, die Bestandsdateien für mehrere Cluster in einem Verzeichnisbaum zu mischen, kann dies zu Problemen führen, wie die BeeGFS HA-Rolle die auf ein bestimmtes Cluster angewendete Konfiguration aggregiert.

> Es ist nicht erforderlich, dass jedes HA-Cluster auf fünf Bausteine skaliert werden kann, bevor ein neues erstellt wird. In vielen Fällen lässt sich das Management mit weniger Bausteinen pro Cluster vereinfachen. Ein Ansatz besteht darin, die Bausteine in jedem einzelnen Rack als HA-Cluster zu konfigurieren.

Empfohlene Prozentsätze für die Überprovisionierung von Storage-Pools

Wenn Sie den standardmäßigen vier Volumes pro Storage-Pool-Konfiguration für Bausteine der zweiten Generation folgen, finden Sie in der folgenden Tabelle.

Diese Tabelle enthält empfohlene Prozentsätze, die als Volume-Größe im verwendet werden können `eseries_storage_pool_configuration` Für jede BeeGFS-Metadaten oder jedes Storage-Ziel:

Laufwerkgröße	Größe
1,92 TB	18
3,84 TB	21.5
7,68 TB	22.5
15,3 TB	24

> Die oben stehende Anleitung gilt nicht für den Speicherpool, der den Management-Service enthält. Dieser sollte die Größe von über 25 % verringern, um 1 % des Speicherpools für Management-Daten zuzuweisen.

Um zu verstehen, wie diese Werte ermittelt wurden, lesen Sie "TR-4800: Anhang A: Verständnis von SSD-Ausdauer und -Überprovisionierung".

Baustein mit hoher Kapazität

Im Standard BeeGFS Solution Deployment Guide werden Verfahren und Empfehlungen für High-Performance-Workload-Anforderungen beschrieben. Kunden, die hohe Kapazitätsanforderungen erfüllen möchten, sollten die hier aufgeführten Variationen bei Implementierung und Empfehlungen beobachten.

[]

Controller

Wenn Sie Bausteine mit hoher Kapazität benötigen, sollten EF600 Controller durch EF300 Controller ersetzt werden, wobei jeweils eine Cascade HIC zur SAS-Erweiterung installiert ist. Jeder Block-Node verfügt über eine minimale Anzahl von NVMe-SSDs, die im Array-Gehäuse für BeeGFS-Metadaten-Storage befüllt sind, und wird an Erweiterungs-Shelfs mit NL-SAS-HDDs für BeeGFS-Storage-Volumes angeschlossen.

Die Konfiguration des Datei-Node zu Block-Nodes bleibt unverändert.

Laufwerkplatzierung

In jedem Block-Node sind mindestens 4 NVMe-SSDs für BeeGFS-Metadaten-Storage erforderlich. Diese Laufwerke sollten in den äußeren Steckplätzen des Gehäuses platziert werden.

[]

Erweiterungsfächer

Der Baustein mit hoher Kapazität kann mit 1-7, 60 Laufwerkserweiterungsfächern pro Speicher-Array dimensioniert werden.

Anweisungen zum Kabelanschluss der einzelnen Erweiterungsfächer finden Sie unter "Siehe EF300-Verkabelung für Laufwerk-Shelfs".

Copyright-Informationen

Copyright © 2024 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

Markeninformationen

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.